

DigitalTwin for MyPalletizer 260 M5

Unity Version



Inhaltsverzeichnis

1. Abstract	1
2. Introduction and Goals	2
3. Architecture Constraints	6
4. Context and Scope	9
5. Solution Strategy	14
6. Building Block View	22
7. Runtime View	32
8. Deployment View	41
9. Cross-cutting Concepts	45
10. Architecture Decisions	50
11. Quality Requirements	55
12. Risks and Technical Debts	60
13. Glossary	62

1. Abstract

Dieses Dokument beschreibt die Architektur des digitalen Zwillings für den MyPalletizer 260 M5. Das System ermöglicht die hybride Steuerung von Hardware und Simulation über eine einheitliche Python-Schnittstelle. Es dient primär Ausbildungszwecken an der Hochschule Luzern (HSLU).

2. Introduction and Goals

Das Projekt **DigitalTwin for MyPalletizer 260 M5** hat zum Ziel, eine realitätsnahe und performante Simulationsumgebung für den MyPalletizer 260 M5 bereitzustellen.

Der Digital Twin soll es ermöglichen, den Roboterarm sowohl virtuell in Unity als auch real über die Hardware mit derselben Python-API zu steuern. Dadurch kann derselbe Programmcode in verschiedenen Betriebsmodi verwendet werden: nur in der Simulation, nur auf dem realen Roboter oder parallel in beiden Umgebungen.

Ein zentraler praktischer Hintergrund ist die eingeschränkte Verfügbarkeit der realen Hardware. An der Hochschule Luzern (HSLU) steht der MyPalletizer nur in einem Laborraum zur Verfügung, der nicht jederzeit zugänglich ist. Zudem sind lediglich zwei Roboter vorhanden. Dadurch können nicht alle Studierenden jederzeit direkt mit der Hardware arbeiten.

Die Simulation reduziert diese Einschränkung. Studierende und Robotik-Interessierte können Bewegungsabläufe entwickeln, Szenarien testen und die Funktionsweise des Roboters nachvollziehen, ohne dauerhaft Zugriff auf die reale Hardware zu benötigen. Gleichzeitig wird das Risiko reduziert, den realen Roboter durch fehlerhafte Bewegungen, falsche Winkel oder ungeeignete Tests zu beschädigen.

Das System soll damit Lernen, Entwickeln und Testen in der Robotik einfacher, sicherer und ortsunabhängiger machen.

2.1. Aufgabenstellung

Das System soll eine Verbindung zwischen Python-Steuerung, Unity-Simulation und realem MyPalletizer 260 M5 herstellen.

Dazu stellt die Python-Seite eine zentrale API bereit, über die Benutzer Roboterbewegungen, End-Effektor-Zustände und Simulationsobjekte steuern können. Intern entscheidet das System anhand des gewählten Betriebsmodus, ob ein Befehl an Unity, an die reale Hardware oder an beide Zielsysteme gesendet wird.

Die Unity-Simulation visualisiert den Roboterarm und die Simulationswelt. Sie ermöglicht das Testen von Szenarien, ohne dass die reale Hardware verfügbar sein muss. Die reale Hardware wird über die Herstellerbibliothek `pymycobot` und eine serielle USB-Verbindung angesteuert.

2.2. Functional Requirements

Das System erfüllt die folgenden funktionalen Anforderungen:

- **Hybride Steuerung** Umschaltung zwischen den Betriebsmodi `virtual`, `real` und `both` über eine zentrale Python-API. Dies wird im Kapitel [Betriebsarten](#) näher beschrieben.
 - `virtual`: Befehle werden nur an die Unity-Simulation gesendet.
 - `real`: Befehle werden nur an den realen Roboter gesendet.
 - `both`: Befehle werden sowohl an die Simulation als auch an die reale Hardware gesendet.

- **Gelenksteuerung**
 - Vorgabe von Gelenkwinkeln für alle 4 Achsen des Roboters
 - Unterstützung von synchronen Bewegungen, bei denen mehrere Gelenke gemeinsam bewegt werden
 - Unterstützung von asynchronen Bewegungen, sofern diese durch die API vorgesehen sind
 - Einstellbare Bewegungsgeschwindigkeit
 - Begrenzung der Eingaben entsprechend zulässiger Wertebereiche
- **End-Effektor-Steuerung**
 - Steuerung eines Greifers, zum Beispiel Öffnen und Schliessen
 - Steuerung einer Vakuumpumpe
 - Auswahl des aktiven Werkzeugs, zum Beispiel Greifer oder Pumpe
 - Weiterleitung der Werkzeugbefehle an Simulation und/oder Hardware abhängig vom Betriebsmodus
- **Dynamische Szenariengestaltung**
 - Erstellen (**spawn**) und Löschen (**delete**) von Objekten zur Laufzeit über Python
 - Zurücksetzen oder Leeren der Simulationswelt über **clear**
 - Unterstützung einfacher 3D-Objekte, zum Beispiel Box, Sphere oder Cylinder
 - Zuweisung von Position, Rotation, Skalierung und Farbe
 - Laden von vordefinierten Szenarien (**Presets**), siehe Kapitel [Objekt- und Szenarienkonzept](#)
- **Visuelles Feedback und Status**
 - Darstellung der Roboterbewegung in der Unity-Simulation
 - Darstellung der Simulationswelt mit Objekten und Werkzeugzuständen
 - Steuerung der LED am realen Roboter
 - Darstellung der Szene aus verschiedenen Kameraperspektiven in Unity
- **Kommunikation**
 - Übertragung von Roboterbefehlen an Unity über UDP-Port **5005**
 - Übertragung von Weltbefehlen an Unity über UDP-Port **5006**
 - Übertragung von Hardwarebefehlen über Serial/USB mit **115200** Baud
 - Verwendung strukturierter JSON-Nachrichten für die Kommunikation mit Unity

2.3. Quality Goals

Die Architektur orientiert sich an folgenden Qualitätszielen:

Ziel	Szenario	Maßnahme
Benutzbarkeit	Studierende sollen Bewegungsabläufe mit wenig Einarbeitungszeit entwickeln können.	Einheitliche Python-API mit verständlichen Klassen wie Robot und World .
Latenz	Ein Bewegungsbefehl soll möglichst schnell in der Simulation sichtbar sein.	Verwendung von UDP für schnelle, verbindungslose Kommunikation zwischen Python und Unity.
Konsistenz	Derselbe Python-Code soll möglichst unverändert für Simulation und reale Hardware verwendet werden können.	Kapselung der Betriebsmodi virtual , real und both innerhalb der API.
Realitätsnähe	Die Simulation soll sich ähnlich wie der reale Roboter verhalten.	Begrenzung der Gelenkwinkel und Geschwindigkeiten entsprechend der Hardware sowie Verwendung eines Robotermodells in Unity.
Sicherheit	Fehlerhafte oder riskante Bewegungsabläufe sollen zuerst ohne Gefahr für die Hardware getestet werden können.	Simulation als Testumgebung vor der Ausführung auf dem realen Roboter.
Wartbarkeit	Neue Funktionen, Werkzeuge oder Weltbefehle sollen nachvollziehbar ergänzt werden können.	Klare Trennung zwischen Robot- und World-Logik sowie modulare Klassenstruktur.
Portabilität	Das System soll auf verschiedenen Betriebssystemen lauffähig sein.	Einsatz von Python und Unity als plattformübergreifende Technologien.
Nachvollziehbarkeit	Kommunikation und Systemverhalten sollen während der Entwicklung einfach überprüfbar sein.	Verwendung lesbarer JSON-Nachrichten und dokumentierter Schnittstellen.

2.4. Nicht-Ziele

Folgende Punkte sind nicht primäres Ziel des aktuellen Projekts:

- **Keine hochpräzise physikalische Simulation** Die Unity-Simulation soll ein plausibles Verhalten darstellen, ersetzt aber keine vollständige physikalische Analyse des realen Roboters.
- **Keine vollständige Kamerasimulation** Die am realen Roboter vorhandene Kamera wird aktuell nicht vollständig simuliert. Kamerabasierte Funktionen müssen deshalb weiterhin mit realer Hardware getestet werden.

- **Kein vollständiges Closed-Loop-Regelungssystem** Das System sendet Soll-Zustände an Simulation und Hardware, wertet jedoch aktuell keine vollständigen Ist-Zustände zur kontinuierlichen Regelung aus.
- **Keine vollständige Ablösung der Herstellerbibliothek** Die reale Hardwareansteuerung bleibt von `pymycobot` abhängig. Diese Abhängigkeit wird gekapselt, aber nicht vollständig ersetzt.

2.5. Stakeholders

Stakeholder	Rolle	Interessen / Erwartungen
Studierende der HSLU	Endnutzer	Einfache Nutzung, verständliche API, sichere Umgebung zum Testen von Bewegungsabläufen ohne Risiko für die Hardware.
Dozierende: Prof. Dr. Jana Koehler	Auftraggeberin / Prüferin	Zuverlässiges Tool für Übungen und Projekte, einfache Reproduzierbarkeit, nachvollziehbare Architektur und gute Dokumentation.
Robotik-Interessierte	Externe Nutzer	Gute Dokumentation, einfache Installation, verständliche Beispiele und Möglichkeit zur Erweiterung des Systems.
Entwickler: Colin Felber	Entwickler	Klare Struktur, wartbarer Code, saubere Dokumentation und erfolgreiche Umsetzung im Rahmen der Studienleistung.
Zukünftige Projektgruppen	Weiterentwickler	Verständliche Architektur, klare Schnittstellen, dokumentierte Entscheidungen und einfacher Einstieg in die Weiterentwicklung.

3. Architecture Constraints

Die Architektur des Digital Twins unterliegt verschiedenen technischen, organisatorischen und konventionellen Rahmenbedingungen. Diese Randbedingungen schränken den Lösungsraum ein und beeinflussen die konkrete Umsetzung des Systems.

3.1. Technische Rahmenbedingungen

Randbedingung	Erläuterung
Programmiersprache	Die Steuerungsschicht ist in Python ab Version 3.12 implementiert. Python wurde gewählt, da die Sprache für viele Nutzer gut verständlich ist und sich für die Entwicklung von Bewegungsabläufen, Testszenarien und Demonstrationen eignet.
Simulations-Engine	Für die Visualisierung und Simulation wird Unity verwendet. Dadurch muss die Kommunikationslogik auf Unity-Seite in C# umgesetzt werden. Zusätzlich sind Unity-spezifische Komponenten wie GameObjects, Prefabs, Skripte und Articulation Bodies zu berücksichtigen.
Kommunikation mit der Hardware	Die Ansteuerung des realen MyPalletizer 260 M5 erfolgt über eine serielle USB-Schnittstelle mit einer Baudrate von 115200 . Die konkrete Hardwarekommunikation wird über die Herstellerbibliothek pymycobot gekapselt.
Kommunikation mit der Simulation	Die Kommunikation zwischen Python und Unity erfolgt über UDP mit JSON-basierten Nachrichten . Die Simulation verwendet getrennte Ports für Roboterbefehle und Weltbefehle. Die externen Schnittstellen werden im Kapitel Externe Schnittstellen näher beschrieben.
Prozesstrennung	Steuerlogik in Python und Simulation in Unity laufen in getrennten Prozessen. Eine direkte Integration von Python-Code in Unity ist nicht vorgesehen. Dadurch entsteht eine lose Kopplung zwischen Steuerung und Darstellung.
Betriebsmodi	Das System muss die Modi virtual , real und both unterstützen. Dadurch ergeben sich unterschiedliche Zielsysteme für denselben API-Aufruf: Simulation, reale Hardware oder beide gleichzeitig.
Hardware-Plattform	Das System soll auf Standard-Laptops mit Windows, Linux oder macOS lauffähig sein. Es wird keine spezialisierte Hardware vorausgesetzt, sofern nur die Simulation verwendet wird. Für den Modus real oder both wird ein physischer MyPalletizer 260 M5 benötigt.

Randbedingung	Erläuterung
Abhängigkeit von Herstellerbibliotheken	Für die reale Hardwareansteuerung ist <code>pymycobot</code> erforderlich. Diese Abhängigkeit begrenzt die Kontrolle über tieferliegende Hardwarefunktionen und kann bei Änderungen der Herstellerbibliothek Anpassungen im System notwendig machen.
CAD- und Modellqualität	Die Qualität der Simulation ist durch die verfügbaren CAD-Modelle und Unity-Modelle begrenzt. Wenn Hersteller-Modelle geometrisch vom realen Roboter abweichen, kann auch die Simulation nicht vollständig identisch zur Hardware sein.
Kameraeinschränkung	Die am realen Roboter vorhandene Kamera wird in der aktuellen Simulation nicht vollständig nachgebildet. Kamerabasierte Funktionen oder Bildverarbeitung können deshalb nicht vollständig innerhalb der Unity-Simulation getestet werden.

3.2. Organisatorische Rahmenbedingungen

- **Open Source und Reproduzierbarkeit** Das Projekt wird über GitHub verwaltet. Alle notwendigen Abhängigkeiten, zum Beispiel `pymycobot`, sollen über Standard-Tools wie `pip` installierbar sein. Die Installation und Nutzung sollen so dokumentiert sein, dass andere Personen das System nachvollziehen und reproduzieren können.
- **Zeitraahmen** Die Entwicklung erfolgt im Rahmen eines Semesters im Modul APPLAI und wird im Modul AI-ROBOTICS weiterentwickelt. Dadurch ist der Projektumfang zeitlich begrenzt und es müssen Prioritäten bei Funktionalität, Genauigkeit und Stabilität gesetzt werden.
- **Verfügbarkeit der Hardware** Die realen Roboter stehen nur in eingeschränkter Stückzahl im Labor zur Verfügung, aktuell zwei Stück. Diese eingeschränkte Verfügbarkeit war ein zentraler Treiber für die Entwicklung der Simulation, damit Bewegungsabläufe auch ohne permanenten Hardwarezugriff getestet werden können.
- **Nutzergruppe** Die Hauptzielgruppe sind Studierende und Robotik-Interessierte, die den MyPalletizer kennenlernen und Bewegungsabläufe entwickeln möchten. Die Programmierung soll sich möglichst ähnlich zur Hersteller-API anfühlen, damit vorhandenes Wissen über die Hersteller-API weiterverwendet werden kann.
- **Wartbarkeit** Die Architektur muss für Studierende und zukünftige Projektgruppen nachvollziehbar bleiben. Deshalb werden einfache, transparente Technologien wie Python, JSON und UDP gegenüber komplexeren Middleware-Lösungen bevorzugt.

3.3. Konventionen und Standards

- **Dokumentationsstandard** Die Architektur wird gemäss `arc42` dokumentiert.
- **Coding Standards**
 - Python: Orientierung an `PEP 8`

- C#: Orientierung an gängigen Unity-Konventionen
 - **Kommunikationsprotokoll**
 - Verwendung eines JSON-basierten Protokolls
 - Versionierung der Nachrichten, aktuell Version **1**
 - Einheitliches Feld **type** zur Identifikation des Befehls
 - Einheitliches Feld **v** zur Angabe der Protokollversion
 - **Namenskonventionen**
 - Python-Klassen verwenden aussagekräftige Namen wie **Robot** und **World**.
 - Protokollbefehle verwenden sprechende Namen wie **move_joints**, **spawn**, **delete** oder **set_end_effector**.
 - Ports und Standardwerte werden dokumentiert und nicht versteckt im Code verwendet.
-

3.4. Technische Annahmen

- Die UDP-Kommunikation erfolgt primär lokal über **localhost** oder innerhalb eines stabilen lokalen Netzwerks.
- Da UDP keine garantierte Zustellung bietet, muss das System mit einzelnen verlorenen Paketen umgehen können.
- Die Simulation muss keine vollständig physikalisch exakte Abbildung liefern, sondern ein plausibles und nachvollziehbares Verhalten darstellen.
- Der Benutzer verfügt über grundlegende Python-Kenntnisse zur Nutzung der API.
- Der reale Roboter ist korrekt angeschlossen, eingeschaltet und über die serielle Schnittstelle erreichbar.
- Die verwendeten UDP-Ports **5005** und **5006** sind auf dem Zielsystem verfügbar oder können bei Bedarf angepasst werden.
- Unity und Python werden als getrennte Anwendungen gestartet und müssen gleichzeitig laufen, wenn die Simulation verwendet wird.

4. Context and Scope

In diesem Kapitel wird das System gegenüber externen Akteuren und angebundenen Nachbarsystemen abgegrenzt. Es wird beschrieben, welche Rolle das Python-System einnimmt, welche externen Komponenten beteiligt sind und über welche Schnittstellen die Kommunikation erfolgt.

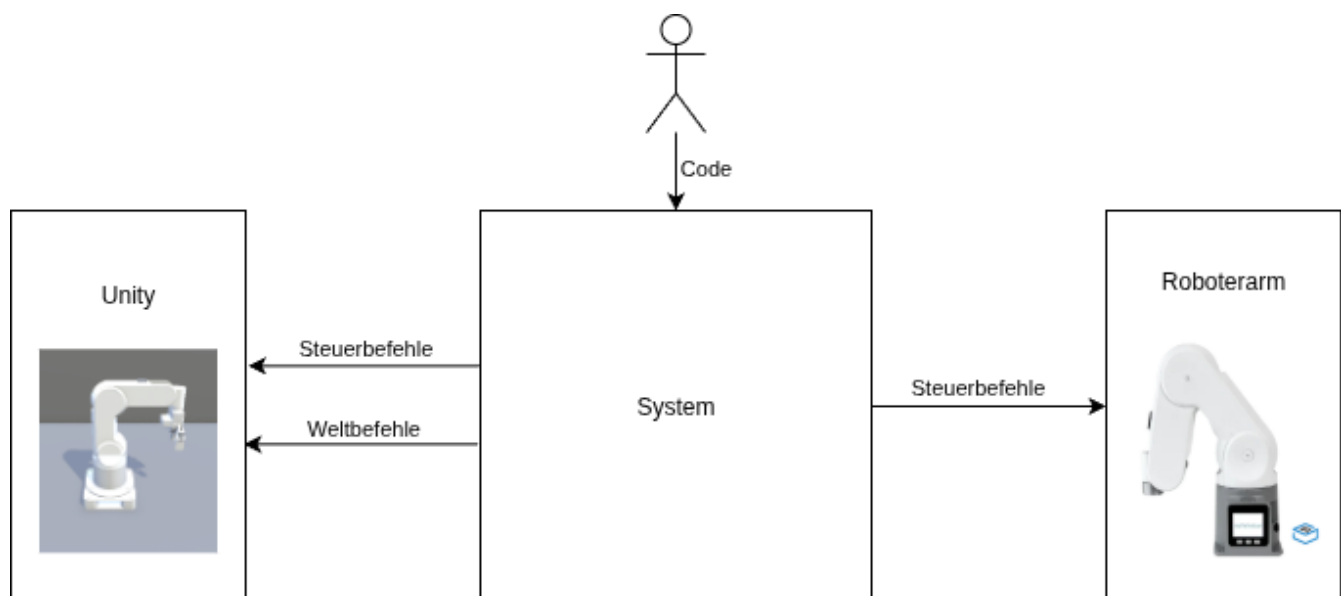
Das betrachtete System ist die Python-basierte Steuerungssoftware. Sie stellt eine einheitliche API bereit, über die Benutzer Roboterbewegungen, Werkzeugbefehle und Weltbefehle auslösen können. Je nach Betriebsmodus werden diese Befehle an die Unity-Simulation, an den realen Roboterarm oder an beide Zielsysteme weitergeleitet.

4.1. Business Context

Der Business Context beschreibt die Nutzung des Systems aus Sicht der Benutzer, zum Beispiel Studierende oder Robotik-Interessierte. Im Vordergrund steht dabei nicht die technische Umsetzung, sondern der typische Ablauf der Nutzung.

Der Benutzer schreibt Python-Code und verwendet dabei die bereitgestellte API. Das System übernimmt die interne Verarbeitung, prüft Eingaben und leitet Befehle an die Simulation (Unity) oder an den realen Roboterarm weiter. Dadurch muss der Benutzer nicht direkt mit Unity, UDP-Nachrichten, serieller Kommunikation oder der Herstellerbibliothek `pymycobot` arbeiten.

Das folgende Diagramm zeigt eine typische Nutzung des Systems.



Rolle / Komponente	Verantwortung im Prozess
Benutzer	Erstellt die Programmlogik in Python und bewertet das Verhalten des Roboters in der Simulation oder auf der realen Hardware.
System	Bildet die zentrale Schnittstelle zwischen Benutzer, Simulation und Hardware. Das System vereinfacht die Nutzung, prüft Eingaben und verteilt Befehle je nach Betriebsmodus an Unity, den realen Roboter oder beide Zielsysteme.

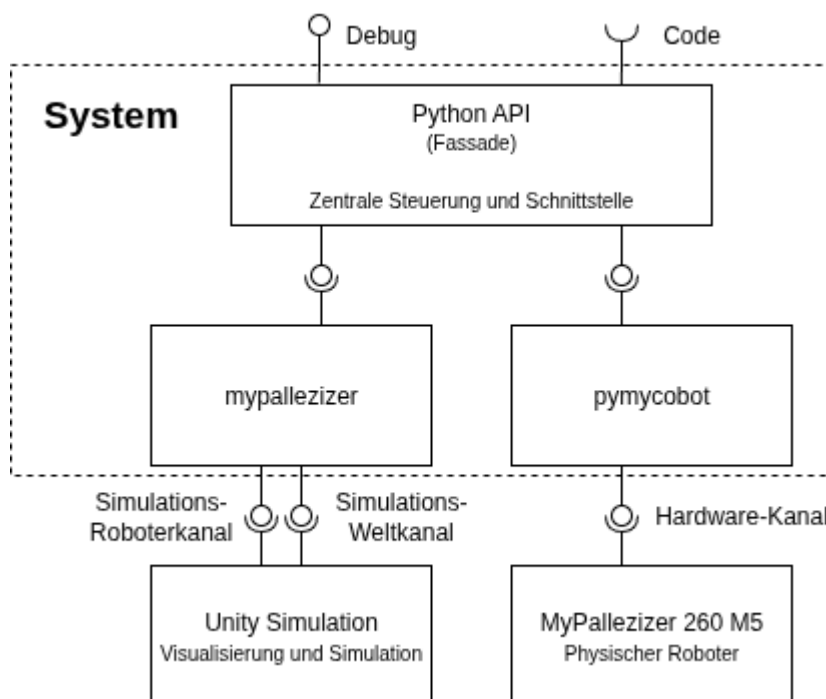
Rolle / Komponente	Verantwortung im Prozess
Unity	Stellt den Roboter und die Simulationsumgebung visuell dar. Unity ermöglicht das Testen von Bewegungen, Werkzeugzuständen und Szenarien ohne zwingenden Einsatz der realen Hardware.
Roboterarm	Führt die Steuerbefehle auf dem realen MyPalletizer 260 M5 aus. Der Roboterarm dient zur realen Überprüfung von Bewegungsabläufen und Hardwarefunktionen.

4.2. Technical Context

Der technische Kontext zeigt die Systemgrenzen sowie die Kommunikationswege zwischen den einzelnen Komponenten.

Im Zentrum steht die Python-API, die als Fassade des Systems dient. Sie nimmt Befehle aus dem Benutzerprogramm entgegen und kapselt die interne Weiterleitung. Für die Simulation werden JSON-Nachrichten über UDP an Unity gesendet. Für die reale Hardware wird die Herstellerbibliothek `pymycobot` verwendet, welche über eine serielle USB-Verbindung mit dem Roboter kommuniziert.

Das folgende Diagramm zeigt die technischen Schnittstellen und die wichtigsten internen Komponenten.



4.2.1. Externe Schnittstellen und beteiligte Komponenten

Schnittstelle / Komponente	Protokoll / Art	Zweck
Debug	Lokal	Ermöglicht das Beobachten, Testen und Nachvollziehen des Systemverhaltens während der Entwicklung.
Code	Lokal (Python)	Der Benutzer schreibt Python-Code, der die bereitgestellte Python-API verwendet. Dadurch können Roboterbewegungen, Werkzeugbefehle und Weltbefehle ausgelöst werden.
Python-API	Lokal (Python)	Bildet die Fassade des Systems. Sie stellt die zentrale Benutzerschnittstelle bereit und kapselt die interne Steuerung, Moduswahl und Weiterleitung der Befehle. Dabei werden die Befehle anhand des eingestellten Betriebsmodus weitergeleitet.
mypalletizer	Internes Python-Modul	Verarbeitet Befehle für die Simulation und stellt die Verbindung zur Unity-Simulation her. Darüber werden sowohl Roboterbefehle als auch Weltbefehle an Unity weitergegeben.
pymycobot	Python-Bibliothek	Wird intern verwendet, um den realen MyPalletizer 260 M5 anzusteuern. Die Bibliothek ist gekapselt, sodass der Benutzer nicht direkt mit ihr arbeiten muss.
Simulations-Roboterkanal	UDP (JSON), Port 5005	Überträgt Steuerbefehle vom System an die Unity-Simulation. Dazu gehören zum Beispiel Gelenkbewegungen, Werkzeugwahl, Greiferzustand oder Pumpensteuerung.
Simulations-Weltkanal	UDP (JSON), Port 5006	Überträgt Weltbefehle vom System an die Unity-Simulation. Dazu gehören zum Beispiel das Erzeugen, Löschen oder Zurücksetzen von Objekten in der Simulationswelt.
Hardware-Kanal	Serial (USB), 115200 Baud	Überträgt Steuerbefehle über pymycobot an den realen MyPalletizer 260 M5.

4.2.2. Abgrenzung des Systems

Innerhalb der Systemgrenze befinden sich die Python-API sowie die internen Module zur Steuerung und Weiterleitung der Befehle. Dazu gehören insbesondere die Logik zur Moduswahl, die Erzeugung der Nachrichten und die Anbindung an Simulation oder Hardware.

Nicht Teil des Systems sind die Unity-Anwendung selbst und der physische Roboterarm. Beide werden als externe Zielsysteme betrachtet, die vom Python-System angesteuert werden.

Die Benutzer arbeiten nicht direkt mit den internen Modulen, UDP-Sockets oder **pymycobot**, sondern ausschliesslich mit der bereitgestellten Python-API.

4.2.3. Betriebsmodi

Das System unterstützt drei Betriebsmodi:

- **virtual** Befehle werden nur an die Unity-Simulation gesendet.
- **real** Befehle werden nur an den realen Roboterarm gesendet.
- **both** Befehle werden sowohl an die Unity-Simulation als auch an den realen Roboterarm gesendet.

Die Moduswahl erfolgt innerhalb der Python-API. Dadurch bleibt der Benutzercode weitgehend gleich, unabhängig davon, ob Simulation, Hardware oder beide Zielsysteme verwendet werden.

4.2.4. Beispielhafte JSON-Nachrichten

Die Kommunikation zwischen Python und Unity erfolgt über strukturierte JSON-Nachrichten. Dabei wird zwischen Roboterbefehlen und Weltbefehlen unterschieden. Beide Nachrichtentypen verwenden eine gemeinsame Grundidee: Jede Nachricht enthält eine Protokollversion **v** sowie ein Feld **type**, das den Befehl eindeutig beschreibt.

Die konkreten Nachrichten werden auf Python-Seite durch das Modul **protocol** erzeugt. Dadurch bleibt der Aufbau der übertragenen Daten zentral definiert und konsistent.

Beispiel einer Nachricht für eine asynchrone Gelenkbewegung

```
{
  "v": 1,
  "type": "move_joints",
  "j1": 0.0,
  "j2": 20.0,
  "j3": -15.0,
  "j4": 10.0,
  "speed": 40
}
```

Diese Nachricht wird über den Roboterkanal an die Simulation übertragen und beschreibt den Soll-Zustand der vier Gelenke sowie die gewünschte Bewegungsgeschwindigkeit.

Beispiel einer Nachricht für die Werkzeugwahl

```
{
  "v": 1,
  "type": "set_end_effector",
  "tool": "gripper"
}
```

Diese Nachricht wählt das aktive Werkzeug des Roboters. Unterstützt werden insbesondere **gripper** und **pump**.

```
{
  "v": 1,
  "type": "spawn",
  "object_type": "box",
  "name": "box_1",
  "position": {
    "x": 0.2,
    "y": 0.0,
    "z": 0.1
  },
  "rotation": {
    "x": 0.0,
    "y": 0.0,
    "z": 0.0
  },
  "scale": {
    "x": 0.05,
    "y": 0.05,
    "z": 0.05
  },
  "color": {
    "r": 255,
    "g": 0,
    "b": 0
  }
}
```

Diese Nachricht wird über den Weltkanal übertragen und beschreibt ein neues Objekt, das in Unity erzeugt werden soll. Neben Typ und Name enthält sie auch Transformations- und Darstellungsparameter wie Position, Rotation, Skalierung und Farbe.

Die Beispiele zeigen bewusst nur einen kleinen Ausschnitt des Protokolls. Weitere Befehle wie `sync_move_joints`, `led`, `set_gripper_state`, `pump`, `delete` und `clear` folgen demselben Grundprinzip.

5. Solution Strategy

Die Lösungsstrategie beschreibt die grundlegenden technischen Ansätze, mit denen die wichtigsten Qualitätsziele des Systems erreicht werden. Im Mittelpunkt stehen eine einfache Benutzbarkeit, eine klare Trennung der Verantwortlichkeiten sowie die Möglichkeit, denselben Python-Code sowohl für die Simulation als auch für den realen Roboter zu verwenden.

Die Architektur basiert auf einer klaren Trennung zwischen der Steuerlogik in Python und der visuellen beziehungsweise physikalischen Darstellung in Unity. Python bildet dabei die zentrale Steuerungs- und Vermittlungsschicht. Unity ist für die Simulation und Visualisierung zuständig, während der reale Roboter über die Herstellerbibliothek `pymycobot` und eine serielle Verbindung angesprochen wird.

Ziel der Lösungsstrategie ist es, die technische Komplexität vor dem Benutzer zu verbergen. Der Benutzer arbeitet mit einer einheitlichen Python-API, ohne direkt mit UDP, serieller Kommunikation, Unity-internen Komponenten oder `pymycobot` interagieren zu müssen.

5.1. Überblick über die Lösungsstrategie

Die zentrale Idee des Systems besteht darin, alle Benutzerbefehle zuerst über eine gemeinsame Python-Schnittstelle entgegenzunehmen. Diese Schnittstelle entscheidet anschliessend, abhängig vom gewählten Modus, an welche Zielsysteme die Befehle weitergeleitet werden.

Es werden drei Betriebsmodi unterstützt:

- **virtual** Befehle werden nur an die Unity-Simulation gesendet. Dieser Modus eignet sich besonders für Tests, Demonstrationen und die Entwicklung ohne reale Hardware.
- **real** Befehle werden nur an den realen MyPalletizer 260 M5 gesendet. Die Kommunikation erfolgt über `pymycobot` und eine serielle USB-Verbindung.
- **both** Befehle werden sowohl an die Unity-Simulation als auch an den realen Roboter gesendet. Dadurch kann das Verhalten der Simulation mit der realen Hardware verglichen werden.

Diese Moduswahl wird intern gekapselt. Dadurch bleibt der Benutzercode weitgehend identisch, unabhängig davon, ob nur simuliert, nur real ausgeführt oder beides parallel verwendet wird.

5.2. Architekturelle Leitprinzipien

1. Einheitliche Abstraktion

Der Benutzer soll nicht unterscheiden müssen, ob er die Simulation oder die reale Hardware steuert.
Alle modusspezifischen Unterschiede (`'virtual'`, `'real'`, `'both'`) werden intern im System behandelt.
Dadurch kann derselbe Python-Code in verschiedenen Ausführungskontexten verwendet

werden.

2. Kapselung externer Abhängigkeiten

Die Herstellerbibliothek `'pymycobot'` wird nicht direkt vom Benutzer verwendet, sondern innerhalb des Systems gekapselt. Dadurch bleibt die öffentliche API stabiler und weniger abhängig von herstellerspezifischen Details. Auch die UDP-Kommunikation mit Unity wird vor dem Benutzer verborgen. Der Benutzer ruft Methoden wie `'send_angles()'`, `'set_color()'` oder `'spawn()'` auf, während das System intern die passenden Nachrichten erzeugt und versendet.

3. Entkopplung durch Netzwerkschnittstellen

Durch die Verwendung von UDP laufen Steuerlogik in Python und Simulation in Unity unabhängig voneinander. Dadurch wird verhindert, dass Performance-Schwankungen der 3D-Engine die Python-Steuerlogik direkt blockieren. Unity kann somit als eigenständiges Simulationssystem betrieben werden, während Python die Rolle der Steuerungsinstanz übernimmt.

4. Zustandsbasierte Kommunikation

Jedes UDP-Paket enthält den vollständigen Soll-Zustand, zum Beispiel alle Gelenkwinkel des Roboters. Dadurch muss die Simulation nicht zwingend jedes einzelne Paket erhalten, um den aktuellen Zielzustand rekonstruieren zu können. Geht ein einzelnes UDP-Paket verloren, kann ein späteres Paket den aktuellen Soll-Zustand erneut übertragen und die Simulation wieder auf den gewünschten Zustand bringen.

5. Trennung von Roboter- und Weltbefehlen

Roboterbefehle und Weltbefehle erfüllen unterschiedliche Aufgaben und haben unterschiedliche Anforderungen. Deshalb werden sie über getrennte UDP-Kanäle übertragen. Roboterbefehle sind häufig und zeitkritisch, während Weltbefehle seltener auftreten und eher diskrete Aktionen darstellen. Die Trennung verbessert die Wartbarkeit und reduziert die Gefahr,

dass grosse oder seltene Weltnachrichten die kontinuierliche Robotersteuerung beeinflussen.

6. Einfache Erweiterbarkeit

Neue Roboterbefehle, Werkzeuge oder Weltbefehle sollen möglichst einfach ergänzt werden können.

Deshalb werden Nachrichten zentral über das Protokollmodul erzeugt und nicht verteilt im Code

manuell zusammengesetzt.

Dadurch bleibt das Nachrichtenformat konsistent und Änderungen am Protokoll können an einer

zentralen Stelle vorgenommen werden.

5.3. Entscheidungen zur Kommunikation

• Warum UDP statt TCP?

In diesem System ist eine geringe Latenz wichtiger als eine garantierte Zustellung. Besonders bei

Bewegungsdaten ist es wichtiger, schnell den aktuellen Zielzustand zu übertragen, als ältere Pakete

nachträglich garantiert auszuliefern.

Ein verlorenes Paket ist in der Regel unkritisch, da kurze Zeit später ein neues Paket mit aktuellen

Daten gesendet werden kann. TCP würde durch Wiederholungen und Reihenfolgesicherung zusätzliche

Verzögerungen verursachen. Gerade für kontinuierliche Bewegungsbefehle kann das zu einer weniger

flüssigen Simulation führen.

UDP ist deshalb für diesen Anwendungsfall geeignet, solange die Nachrichten zustandsbasiert aufgebaut

sind und nicht jede einzelne Nachricht zwingend verarbeitet werden muss.

• Warum JSON?

JSON ist einfach zu implementieren, gut lesbar und mit Python sowie Unity problemlos verarbeitbar.

Das erleichtert insbesondere das Debugging, da Nachrichten direkt gelesen und verstanden werden können.

Da die Datenmenge pro Nachricht klein ist, ist der zusätzliche Overhead gegenüber binären Formaten

akzeptabel. Ein binäres Protokoll wäre zwar effizienter, aber deutlich schwieriger zu debuggen und

weniger transparent.

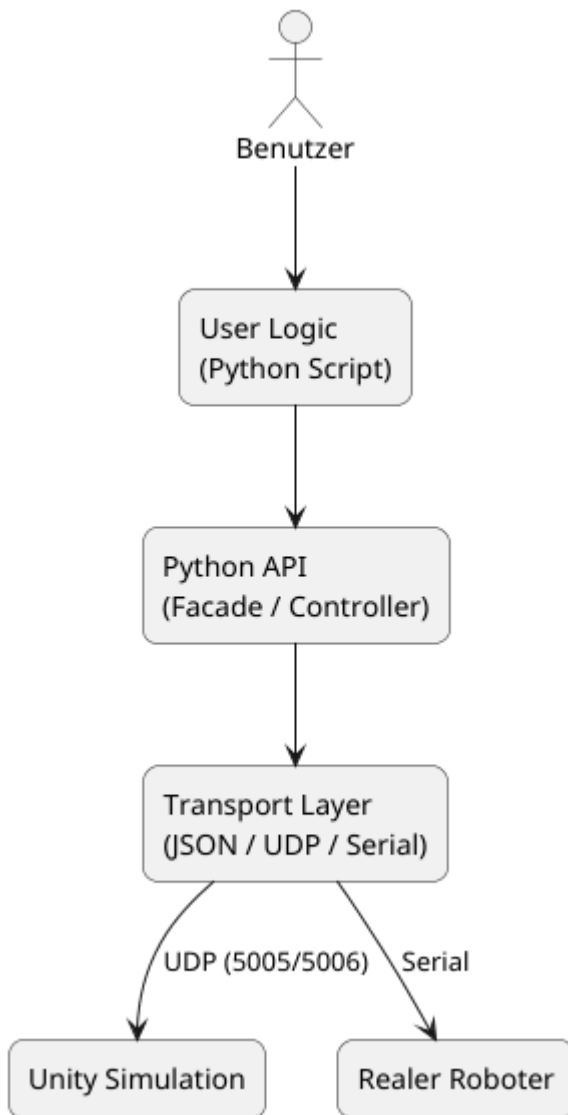
- **Warum Serial für die Hardware?**

Der reale MyPalletizer 260 M5 wird über eine serielle Verbindung angesprochen, da dies dem vorgesehenen Kommunikationsweg der Herstellerbibliothek `'pymycobot'` entspricht. Die Python-Seite nutzt diese Verbindung jedoch nicht direkt im Benutzercode, sondern kapselt sie im Hardware-Controller.

- **Warum zwei UDP-Ports?**

Die Simulation unterscheidet zwischen Roboterbefehlen und Weltbefehlen. Port `'5005'` wird für Roboterbefehle verwendet, Port `'5006'` für Weltbefehle. Dadurch können beide Arten von Nachrichten getrennt verarbeitet werden. Diese Aufteilung vereinfacht die Struktur auf Unity-Seite und macht die Kommunikation robuster gegen gegenseitige Beeinflussung.

5.4. Schichtenmodell



Schicht	Verantwortung
User Logic	Python-Skript des Benutzers. Enthält Bewegungslogik, Testszenarien oder Demonstrationen. Der Benutzer arbeitet auf dieser Ebene mit den Klassen Robot und World .
API / Controller Layer	Zentrale Schnittstelle des Systems. Kapselt die Nutzung von pymycobot , validiert Eingaben und entscheidet anhand des Modus, ob Befehle an Simulation, Hardware oder beide gesendet werden.
Protocol Layer	Erzeugt strukturierte Nachrichten für die Kommunikation mit Unity. Dadurch wird sichergestellt, dass alle JSON-Nachrichten einheitlich aufgebaut sind und Änderungen am Protokoll zentral vorgenommen werden können.
Transport Layer	Verantwortlich für die tatsächliche Übertragung der Daten. Für die Simulation werden JSON-Nachrichten über UDP übertragen. Für die reale Hardware wird die serielle Verbindung über pymycobot verwendet.
Execution Layer	Führt die Befehle aus. Dazu gehören die Unity-Simulation sowie der reale MyPalletizer 260 M5.

5.5. Strategie zur Moduswahl

Die Moduswahl ist ein zentraler Bestandteil der Lösungsstrategie. Sie bestimmt, wohin ein Befehl weitergeleitet wird.

Im Modus **virtual** werden Befehle ausschliesslich an Unity gesendet. Dadurch kann der Benutzer das System ohne reale Hardware testen. Dieser Modus ist besonders nützlich für Entwicklung, Unterricht, Demonstrationen und erste Tests von Bewegungsabläufen.

Im Modus **real** werden Befehle ausschliesslich an den realen Roboter gesendet. Dieser Modus wird verwendet, wenn Bewegungen tatsächlich auf der Hardware ausgeführt werden sollen.

Im Modus **both** werden Befehle parallel an Unity und an den realen Roboter gesendet. Dieser Modus ermöglicht einen direkten Vergleich zwischen Simulation und realem Verhalten. Dabei ist jedoch zu beachten, dass Simulation und Hardware nicht zwingend exakt synchron reagieren, da beide Systeme unterschiedliche Laufzeiten und physikalische Eigenschaften besitzen.

Die Moduswahl wird nicht dem Benutzerprogramm überlassen, sondern zentral in der API behandelt. Dadurch wird verhindert, dass der Benutzercode mit vielen Fallunterscheidungen belastet wird.

5.6. Strategie zur Kommunikationstrennung

Die Kommunikation zwischen Python und Unity erfolgt über zwei getrennte UDP-Ports. Port **5005** wird für Roboterbefehle wie Gelenkbewegungen, Werkzeugbefehle und LED-Steuerung verwendet. Port **5006** wird für Befehle zur Verwaltung der Simulationswelt eingesetzt, beispielsweise für **spawn**, **delete** oder **clear**.

Diese Ports sind standardmässig festgelegt, können aber bei Bedarf angepasst werden.

Diese Trennung wurde bewusst gewählt, da beide Kommunikationsarten unterschiedliche Anforderungen haben. Roboterbefehle treten häufig auf und sind zeitkritisch, da Bewegungen möglichst flüssig und ohne Verzögerung in der Simulation sichtbar werden sollen. Befehle zur Weltverwaltung sind dagegen seltener und eher diskrete Einzelaktionen.

Durch die Aufteilung auf zwei getrennte Kanäle wird verhindert, dass grössere oder unregelmässige Weltnachrichten die kontinuierliche Übertragung der Roboterbefehle beeinflussen.

Zusätzlich verbessert diese Trennung die Struktur des Systems, da Robotersteuerung und Weltverwaltung auch konzeptionell als zwei unterschiedliche Aufgabenbereiche behandelt werden. Dadurch bleibt die Architektur klarer, leichter erweiterbar und einfacher zu warten.

5.7. Strategie zur Fehlerbehandlung

Das System verwendet derzeit überwiegend eine einfache Fehlerbehandlung. Eingaben werden auf Python-Seite geprüft, bevor sie an Simulation oder Hardware weitergeleitet werden. Dadurch können offensichtliche Fehler, zum Beispiel ungültige Gelenkwinkel oder unbekannte Werkzeugnamen, frühzeitig abgefangen werden.

Bei der Kommunikation mit Unity wird bewusst auf ein einfaches UDP-basiertes Modell gesetzt. Da UDP keinen eingebauten Rückkanal und keine garantierte Zustellung besitzt, wird die Robustheit nicht über Paketbestätigung erreicht, sondern über zustandsbasierte Nachrichten.

Für die Hardwaresteuerung ist die Fehlerbehandlung stärker von `pymycobot` und der seriellen Verbindung abhängig. Fehler wie nicht erreichbare Ports, Verbindungsprobleme oder nicht reagierende Hardware müssen auf dieser Ebene erkannt und behandelt werden.

Eine mögliche spätere Erweiterung wäre ein einheitliches Status- und Fehlermodell, bei dem sowohl Unity als auch der reale Roboter Rückmeldungen an Python senden können.

5.8. Strategie zur Erweiterbarkeit

Die Architektur ist so aufgebaut, dass neue Funktionen schrittweise ergänzt werden können.

Neue Roboterbefehle können in der Python-API ergänzt und anschliessend im Protokollmodul als neue JSON-Nachrichten definiert werden. Unity muss diese Nachrichtentypen danach entsprechend interpretieren.

Neue Weltbefehle können ebenfalls über das Protokollmodul ergänzt werden. Beispiele wären zusätzliche Objekttypen, komplexere Szenenoperationen oder erweiterte Material- und Farbeinstellungen.

Auch weitere Hardwarefunktionen können ergänzt werden, solange sie entweder über `pymycobot` verfügbar sind oder über eine eigene Hardware-Abstraktionsschicht angebunden werden.

Wichtig ist dabei, dass die öffentliche API möglichst stabil bleibt. Änderungen an internen Kommunikationswegen sollen den Benutzercode möglichst wenig beeinflussen.

5.9. Bewusste technische Einschränkungen

Einige technische Vereinfachungen wurden bewusst akzeptiert, um das System überschaubar und für den Projektumfang realistisch zu halten.

- **Kein vollständiges Closed-Loop-System**

Die Python-Seite sendet Soll-Zustände, prüft aber aktuell nicht kontinuierlich, ob Simulation oder

Hardware diese Zustände exakt erreicht haben. Dadurch bleibt die Architektur einfacher, allerdings fehlen präzise Rückmeldungen über den tatsächlichen Zustand.

- **Keine vollständige physikalische Genauigkeit**

Die Unity-Simulation dient vor allem der Visualisierung, Demonstration und grundlegenden Prüfung von Bewegungsabläufen. Sie ersetzt keine hochpräzise physikalische Simulation des realen Roboters.

- **Keine vollständige Kamerasimulation**

Die am realen Roboter vorhandene Kamera wird in der aktuellen Simulation nicht vollständig nachgebildet. Kamerabasierte Funktionen oder Bildverarbeitung müssen daher weiterhin mit der echten Hardware getestet werden.

- **Abhängigkeit von Herstellerkomponenten**

Die reale Hardwareansteuerung hängt weiterhin von 'pymycobot' und den Fähigkeiten des Herstellersystems ab. Diese Abhängigkeit wird gekapselt, aber nicht vollständig eliminiert.

6. Building Block View

Die Bausteinsicht beschreibt die statische Struktur des Systems. Sie zeigt, aus welchen Komponenten das System besteht, welche Verantwortlichkeiten diese Komponenten besitzen und wie sie miteinander zusammenarbeiten.

Dabei werden sowohl die Python-Seite als auch die Unity-Seite betrachtet. Zusätzlich wird dargestellt, wie der reale MyPalletizer 260 M5 und die Herstellerbibliothek `pymycobot` in die Architektur eingebunden sind.

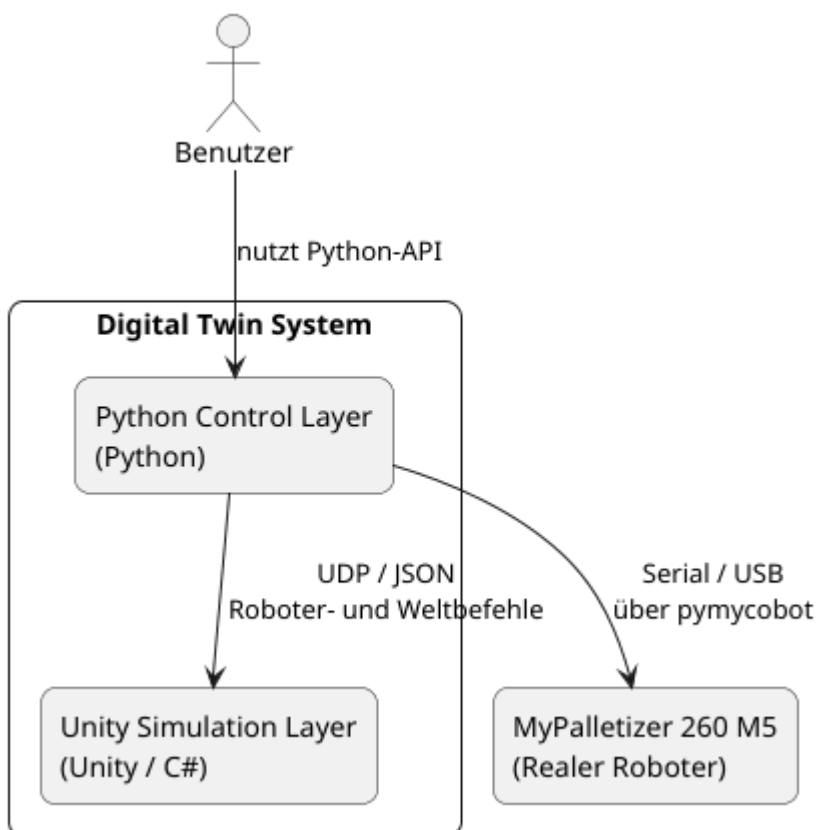
Das Ziel der Bausteinsicht ist es, die interne Struktur des Systems nachvollziehbar zu machen. Insbesondere wird gezeigt, wie die einheitliche Python-API von der konkreten Ausführung in Simulation und Hardware entkoppelt wird.

6.1. Whitebox Overall System (Level 1)

Auf der obersten Ebene besteht das System aus einer Python-basierten Steuerungs- und API-Schicht sowie einer Unity-basierten Simulationsschicht. Der reale Roboter ist ein externes Zielsystem, das über die Python-Seite angebunden wird.

Die Python-Schicht bildet die zentrale Schnittstelle für den Benutzer. Sie nimmt API-Aufrufe entgegen, validiert Eingaben und entscheidet anhand des gewählten Betriebsmodus, ob ein Befehl an die Simulation, an den realen Roboter oder an beide Zielsysteme weitergeleitet wird.

Die Unity-Schicht visualisiert den Roboter und die Simulationswelt. Sie empfängt JSON-Nachrichten über UDP und setzt diese in Bewegungen, Werkzeugzustände oder Änderungen der Simulationswelt um.



6.1.1. Enthaltene Bausteine

Baustein	Beschreibung
Python Control Layer	Stellt die zentrale Schnittstelle für den Benutzer bereit. Diese Schicht kapselt die Unterschiede zwischen Simulation, realer Hardware und kombiniertem Betrieb. Sie enthält unter anderem die Klassen Robot , World , den MyPalletizerController sowie Hilfsmodule für Protokoll, Presets und Objekte.
Unity Simulation Layer	Empfängt Befehle aus Python und setzt diese in simulierte Bewegungen, Werkzeugfunktionen und Änderungen der Simulationswelt um. Die Unity-Schicht enthält getrennte Komponenten für Roboterbefehle und Weltbefehle.
Realer Roboter: MyPalletizer 260 M5	Führt Befehle auf der physischen Hardware aus. Die Ansteuerung erfolgt über die Herstellerbibliothek pymycobot , welche innerhalb der Python-Schicht gekapselt ist.
Benutzer	Schreibt Python-Code und verwendet die bereitgestellte API. Der Benutzer arbeitet nicht direkt mit UDP, Unity-internen Komponenten oder pymycobot .

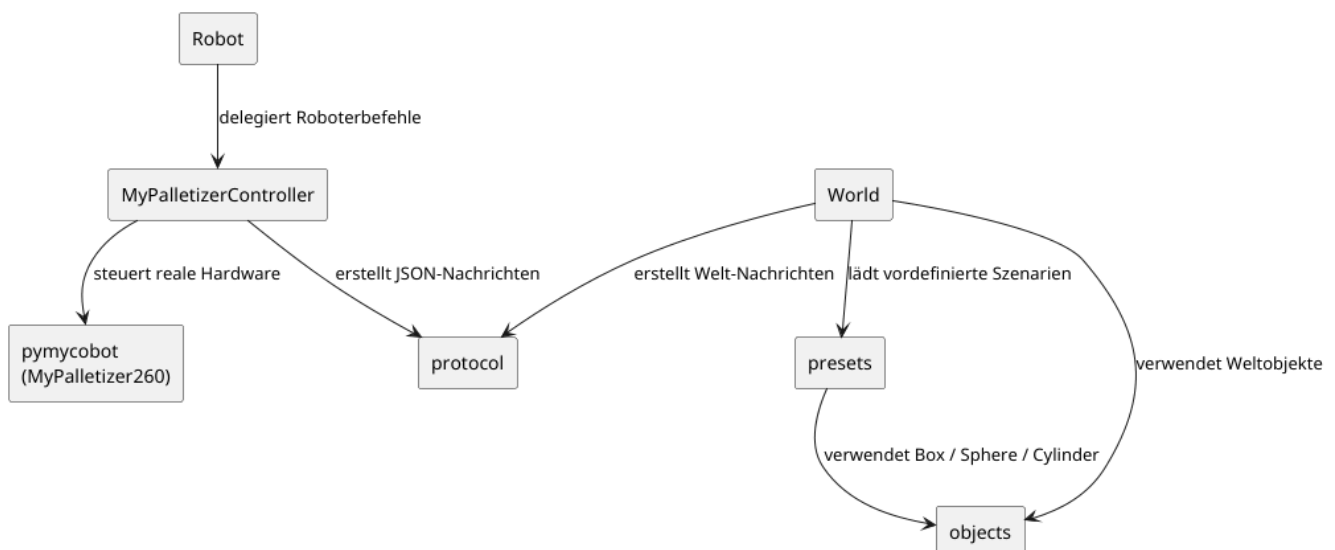
6.2. Level 2

Auf Level 2 werden die internen Bausteine der Python- und Unity-Schicht genauer beschrieben.

6.2.1. Whitebox Python Control Layer

Die Python-Schicht ist so aufgebaut, dass der Benutzer mit einer einfachen API arbeitet, während die eigentliche Verteilung auf Simulation und Hardware intern erfolgt.

Die beiden wichtigsten Einstiegspunkte für den Benutzer sind **Robot** und **World**. **Robot** ist für Roboterbewegungen, Werkzeuge und LED-Steuerung zuständig. **World** ist für Simulationsobjekte und Szenarien zuständig.



Beschreibung der Bausteine

Baustein	Beschreibung
Robot	Benutzernahe API für die Robotersteuerung. Diese Klasse kapselt die Nutzung des Controllers und bietet einfache Methoden wie <code>send_angles()</code> , <code>sync_move_joints()</code> , <code>set_color()</code> , <code>set_gripper_state()</code> und <code>pump_on()</code> . Zudem validiert sie die Grundkonfiguration, zum Beispiel den benötigten Serial-Port in den Modi <code>real</code> und <code>both</code> .
MyPalletizerController	Zentrale Steuerkomponente für den Roboter. Diese Klasse entscheidet anhand des Modus <code>virtual</code> , <code>real</code> oder <code>both</code> , ob ein Befehl an die Simulation, an den realen Roboter oder an beide weitergeleitet wird. Zusätzlich begrenzt sie Gelenkwinkel und Geschwindigkeit und verwaltet den aktuellen Simulationszustand der Gelenke.
World	Benutzernahe API für die Verwaltung der Simulationswelt. Diese Klasse ist bewusst von der Robotersteuerung getrennt und bietet Funktionen wie <code>spawn()</code> , <code>delete()</code> , <code>clear()</code> , <code>list_presets()</code> und <code>load_preset()</code> .
protocol	Hilfsmodul zum Aufbau der JSON-Nachrichten. Es definiert sowohl Roboterbefehle wie <code>move_joints</code> , <code>sync_move_joints</code> , <code>led</code> , <code>set_gripper_state</code> und <code>pump</code> als auch Weltbefehle wie <code>spawn</code> , <code>delete</code> und <code>clear</code> .
presets	Enthält vordefinierte Szenarien für die Simulation, zum Beispiel <code>empty</code> , <code>three_blocks</code> , <code>pick_and_place_basic</code> und <code>sorting_task</code> . Presets ermöglichen wiederholbare Test- und Demonstrationsszenarien.
objects	Definiert einfache Weltobjekte wie <code>Box</code> , <code>Sphere</code> und <code>Cylinder</code> . Diese Objekte können durch die <code>World</code> -Klasse in die Simulation eingebracht werden und enthalten Informationen zu Position, Rotation, Skalierung und Farbe.
pymycobot	Externe Herstellerbibliothek zur Ansteuerung des realen MyPalletizer. Sie wird nicht direkt vom Benutzer verwendet, sondern ausschliesslich intern über den <code>MyPalletizerController</code> .

Architekturidee der Python-Schicht

Wie bereits im Kapitel [Strategie zur Kommunikationstrennung](#) beschrieben, trennt die Python-Schicht bewusst zwischen Robotersteuerung und Weltverwaltung.

`Robot` und `World` bilden zwei eigenständige Zugriffspunkte für den Benutzer. Intern übernimmt der `MyPalletizerController` die Orchestrierung der Roboterbefehle, während `World` einen separaten Kommunikationsweg für Weltänderungen verwendet.

Dadurch ergibt sich eine klare Aufteilung:

- **Robot** für Bewegungen, Werkzeuge und LED
- **World** für Objekte und Szenarien
- **protocol** für die gemeinsame Nachrichtenstruktur
- **presets** für wiederholbare Szenarien

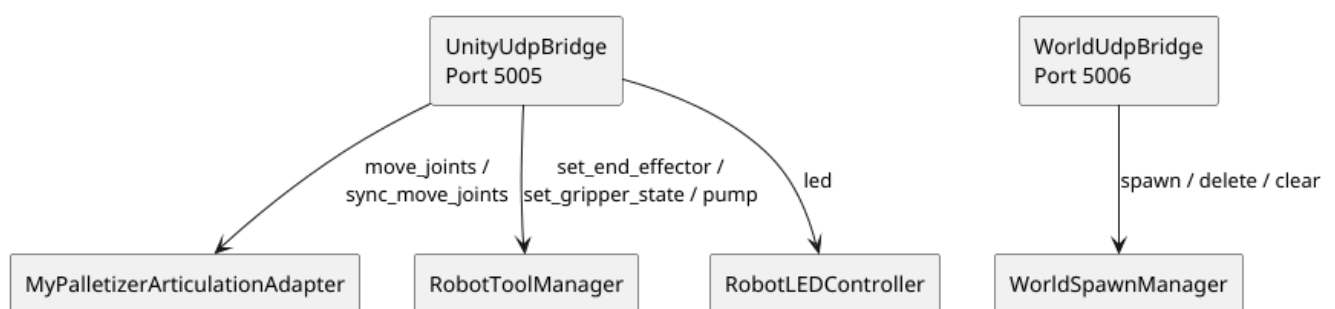
- **objects** für die Beschreibung von Weltobjekten
- **pymycobot** für die reale Hardwareanbindung

Diese Trennung reduziert Abhängigkeiten zwischen den Bausteinen und erleichtert spätere Erweiterungen.

6.2.2. Whitebox Unity Simulation Layer

Die Unity-Schicht ist ebenfalls in zwei Hauptbereiche getrennt: Robotersteuerung und Weltverwaltung. Dadurch werden zeitkritische Roboterbefehle nicht mit Weltänderungen vermischt.

Roboterbefehle werden über den Port **5005** empfangen. Weltbefehle werden über den Port **5006** empfangen.



Beschreibung der Bausteine

Baustein	Beschreibung
UnityUdpBridge	Empfängt Roboterbefehle über UDP auf Port 5005 . Die empfangenen JSON-Nachrichten werden zunächst in eine Queue gelegt und anschliessend im Unity-Hauptthread verarbeitet. Dadurch wird verhindert, dass Unity-Objekte direkt aus einem Netzwerk-Thread verändert werden.
MyPalletizerArticulationAdapter	Übersetzt Gelenkbefehle in konkrete Bewegungen der vier simulierten Gelenke. Dabei werden sowohl asynchrone als auch synchrone Bewegungen unterstützt. Die eigentliche Gelenkbewegung wird an einzelne ArticulationJointActuator -Komponenten delegiert.
RobotToolManager	Verwaltet die aktiven Werkzeuge des Roboters innerhalb der Simulation. Diese Komponente reagiert auf Befehle zum Werkzeugwechsel sowie auf Greifer- und Pumpenbefehle.
RobotLEDController	Setzt empfangene RGB-Werte in die visuelle LED-Darstellung des Roboters um.
WorldUdpBridge	Empfängt Weltbefehle über UDP auf Port 5006 . Auch hier werden Nachrichten zunächst zwischengespeichert und anschliessend sicher im Unity-Hauptthread verarbeitet.

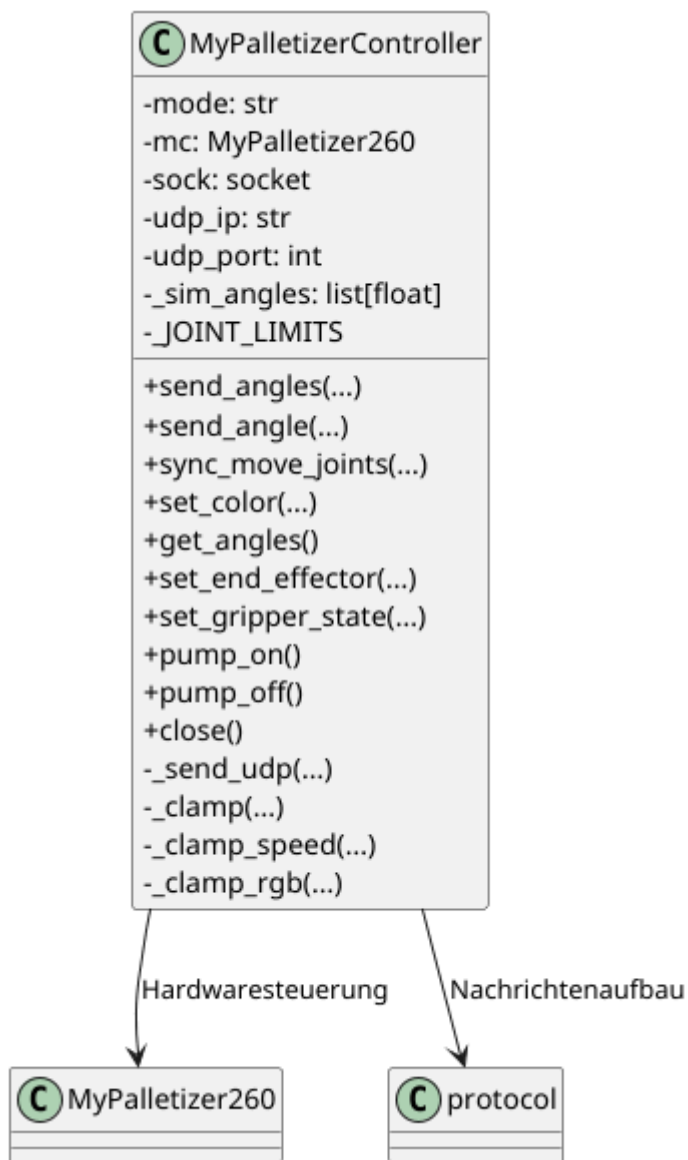
Baustein	Beschreibung
WorldSpawnManager	Verarbeitet die Weltbefehle spawn , delete und clear . Er erstellt Objekte, entfernt sie wieder oder setzt die Szene zurück. Beim Erzeugen von Objekten ergänzt er benötigte Unity-Komponenten wie Rigidbody und GrabbableObject .

6.3. Level 3

Auf Level 3 werden die wichtigsten internen Strukturen der Python- und Unity-Schicht detaillierter betrachtet.

6.3.1. Whitebox **MyPalletizerController**

Der **MyPalletizerController** ist der zentrale technische Kern der Python-Schicht. Er verbindet die benutzernahe API mit der Simulation und der realen Hardware.



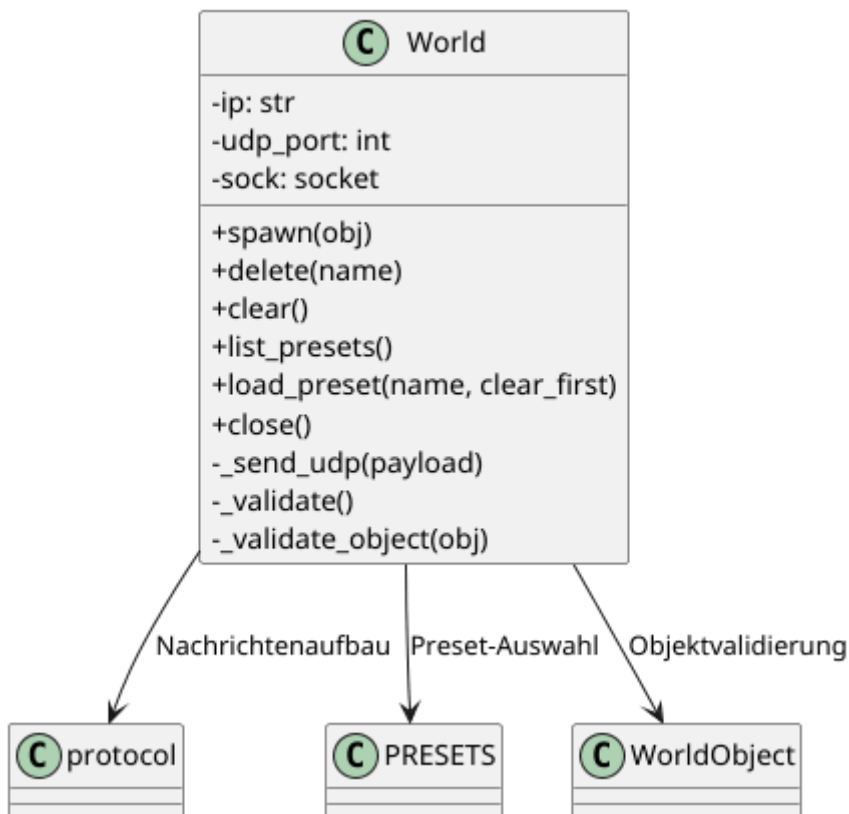
Aufgaben des Controllers

Der `MyPalletizerController` erfüllt mehrere Aufgaben:

- **Modusabhängige Verteilung** Im Modus `virtual` werden Befehle nur per UDP an Unity gesendet. Im Modus `real` werden Befehle nur über `pymycobot` an den echten Roboter gesendet. Im Modus `both` werden beide Wege parallel verwendet.
- **Validierung und Begrenzung** Gelenkwinkel werden auf feste Grenzwerte begrenzt. Auch die Geschwindigkeit wird in einen erlaubten Bereich gebracht. RGB-Werte für die LED werden ebenfalls geprüft und begrenzt.
- **Verwaltung des Simulationszustands** Für den virtuellen Betrieb werden Gelenkwinkel intern in `_sim_angles` gespeichert. Dadurch kann `get_angles()` auch ohne reale Hardware einen sinnvollen Zustand liefern.
- **Kapselung der Herstellerbibliothek** `pymycobot` bleibt vollständig hinter dem Controller verborgen. Der Benutzer arbeitet nur mit der `Robot`-Klasse und nicht direkt mit der Hersteller-API.
- **Kommunikation mit Unity** Roboterbefehle werden in JSON-Nachrichten umgewandelt und über UDP an die Unity-Simulation gesendet.

6.3.2. Whitebox `World`

Die Klasse `World` ist der zweite zentrale Zugangspunkt auf Python-Seite. Sie ist bewusst unabhängig von der Robotersteuerung.



Aufgaben der World-Komponente

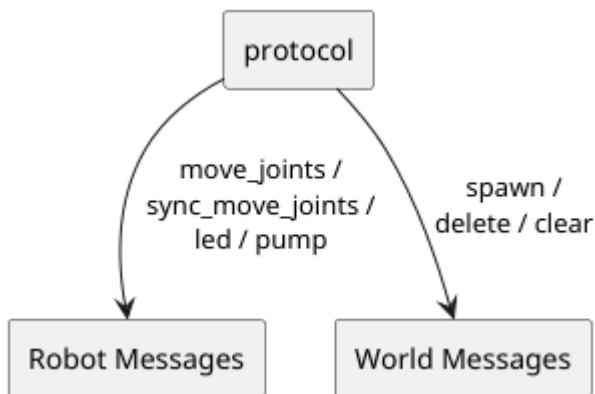
World kapselt alle Befehle, die sich auf die Simulationswelt beziehen. Dazu gehören:

- Erzeugen neuer Objekte
- Löschen vorhandener Objekte
- Zurücksetzen der Szene
- Laden von vordefinierten Presets
- Validieren von Weltobjekten vor dem Versand an Unity

Die Trennung von **World** und **Robot** sorgt dafür, dass Weltänderungen die eigentliche Robotersteuerung nicht unnötig vermischen.

6.3.3. Whitebox **protocol**

Das Modul **protocol** ist kein eigenständiger Laufzeitprozess, sondern ein Hilfsmodul zur zentralen Erzeugung von Nachrichten.



Aufgaben des Protokollmoduls

Das Protokollmodul sorgt dafür, dass JSON-Nachrichten konsistent aufgebaut werden. Dadurch müssen Nachrichten nicht an vielen Stellen im Code manuell zusammengesetzt werden.

Es übernimmt insbesondere:

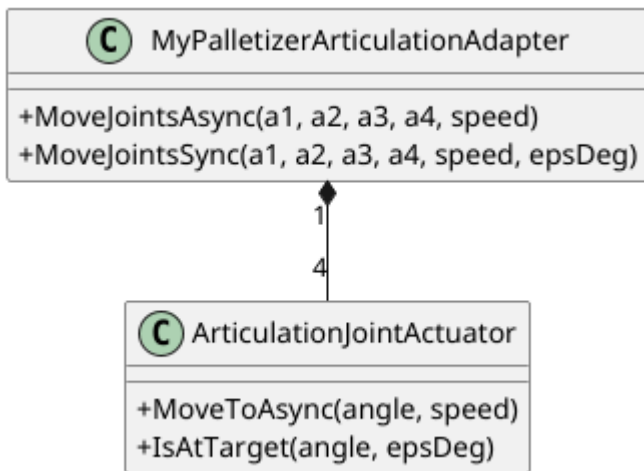
- Aufbau von Roboterbefehlen
- Aufbau von Weltbefehlen
- Verwendung einer einheitlichen Protokollversion **v**
- Verwendung eines einheitlichen Befehlsfeldes **type**
- Reduktion von Fehlern durch zentral definierte Nachrichtenstruktur

Diese Zentralisierung erleichtert spätere Erweiterungen des Protokolls.

6.3.4. Whitebox **MyPalletizerArticulationAdapter**

Innerhalb der Unity-Simulation übernimmt der **MyPalletizerArticulationAdapter** die Weitergabe

von Gelenkbefehlen an die einzelnen Gelenkaktuatoren.



Aufgaben des Adapters

Der Adapter verteilt die empfangenen Zielwinkel auf vier Gelenkaktuatoren.

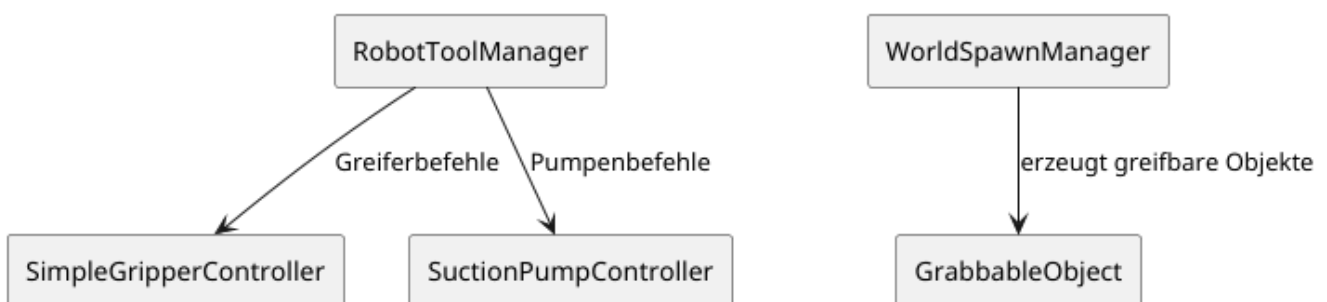
Dabei gibt es zwei Ausführungsarten:

- **asynchron** über `MoveJointsAsync(...)`
- **synchron** über `MoveJointsSync(...)`, wobei gewartet wird, bis alle Gelenke ihr Ziel erreicht haben

Damit bildet der Adapter die Gelenkbewegung des Roboters in Unity auf einer klaren, gekapselten Ebene ab.

6.3.5. Whitebox Werkzeug- und Weltverwaltung in Unity

Neben den Gelenken besitzt die Unity-Seite weitere spezialisierte Komponenten für Werkzeuge und Weltobjekte.



Aufgaben dieser Komponenten

- **RobotToolManager** Koordiniert Werkzeugwechsel und Werkzeugbefehle innerhalb der Simulation. Er entscheidet, ob der Greifer oder die Pumpe aktiv ist.
- **SimpleGripperController** Greift Objekte innerhalb eines definierten Bereichs, hält sie fest und gibt sie wieder frei. Damit wird das Verhalten eines einfachen Greifers simuliert.
- **SuctionPumpController** Aktiviert oder deaktiviert die Pumpe und bindet das nächstgelegene

greifbare Objekt an den Roboter. Dadurch wird das Verhalten einer einfachen Vakuumpumpe in der Simulation nachgebildet.

- **WorldSpawnManager** Erstellt Objekte anhand der empfangenen Weltnachrichten. Zusätzlich ergänzt er benötigte Komponenten wie **Rigidbody** und **GrabbableObject**.
- **GrabbableObject** Markiert Objekte, die durch Greifer oder Pumpe aufgenommen werden können. Dadurch müssen Werkzeugkomponenten nicht jedes beliebige Unity-Objekt berücksichtigen, sondern nur Objekte mit dieser Markierung.

6.4. Wichtige Entwurfsentscheidungen in der Bausteinsicht

Die Bausteinsicht zeigt mehrere wichtige Strukturentscheidungen des Systems:

1. Trennung von Robotersteuerung und Weltverwaltung

Auf Python-Seite sind diese Verantwortlichkeiten durch `'Robot'` und `'World'` getrennt.
Auf Unity-Seite erfolgt die Trennung durch `'UnityUdpBridge'` und `'WorldUdpBridge'`.
Diese Aufteilung verhindert, dass Bewegungsbefehle und Weltbefehle unnötig vermischt werden.

2. Kapselung externer Abhängigkeiten

Die Herstellerbibliothek `'pymycobot'` wird nicht direkt an den Benutzer weitergegeben.
Sie ist vollständig über den `'MyPalletizerController'` und die Klasse `'Robot'` gekapselt.
Dadurch bleibt die öffentliche API stabiler und unabhängiger von herstellerspezifischen Details.

3. Symmetrische Struktur zwischen Python und Unity

Auf beiden Seiten gibt es eine getrennte Verarbeitung von Roboter- und Weltbefehlen.
Das vereinfacht die Architektur und macht die Kommunikation klarer.

4. Zentrale Protokollerzeugung

JSON-Nachrichten werden nicht verteilt im Code aufgebaut, sondern zentral über das Modul `'protocol'`.
Dadurch bleibt das Nachrichtenformat konsistent und leichter erweiterbar.

5. Unity-Hauptthread als Ausführungspunkt

Eingehende UDP-Nachrichten werden in Unity zunächst zwischengespeichert. Die eigentliche Verarbeitung erfolgt anschliessend im Unity-Hauptthread. Dadurch werden Threading-Probleme vermieden, da Unity-Objekte nicht direkt aus Netzwerk-Threads verändert werden sollen.

7. Runtime View

Die Laufzeitsicht zeigt, wie die einzelnen Bausteine des Systems zur Laufzeit zusammenarbeiten. Dabei ist wichtig, dass das System je nach Betriebsmodus (**virtual**, **real**, **both**) unterschiedliche Ausführungspfade verwendet.

Die Python-Seite übernimmt die Orchestrierung. Sie nimmt Befehle über die Benutzer-API entgegen, validiert Eingaben, erzeugt bei Bedarf JSON-Nachrichten und leitet Befehle an die passende Zielkomponente weiter. Die eigentliche Ausführung erfolgt je nach Betriebsmodus entweder in der Unity-Simulation, auf dem realen Roboter oder auf beiden Systemen gleichzeitig.

7.1. Überblick

Die wichtigsten Laufzeitszenarien des Systems sind:

- asynchrone Roboterbewegung in der Simulation
- synchrone Roboterbewegung in der Simulation
- Roboterbewegung auf realer Hardware
- gleichzeitige Ausführung in Simulation und Hardware
- Werkzeug- und End-Effektor-Steuerung
- Verwaltung der Simulationswelt
- typische Fehler- und Grenzfälle

Diese Szenarien werden in den folgenden Abschnitten beschrieben.

7.2. Grundprinzip der Laufzeitverarbeitung

Alle Benutzerbefehle folgen demselben Grundprinzip:

1. Der Benutzer ruft eine Methode der Python-API auf.
2. Die API delegiert den Befehl an die zuständige Komponente.
3. Eingaben werden validiert und bei Bedarf begrenzt.
4. Je nach Betriebsmodus wird der Befehl an Unity, an die reale Hardware oder an beide Zielsysteme gesendet.
5. Unity oder der reale Roboter führen den Befehl aus.

Für Roboterbefehle ist die Klasse **Robot** der Einstiegspunkt. Für Weltbefehle ist die Klasse **World** der Einstiegspunkt.

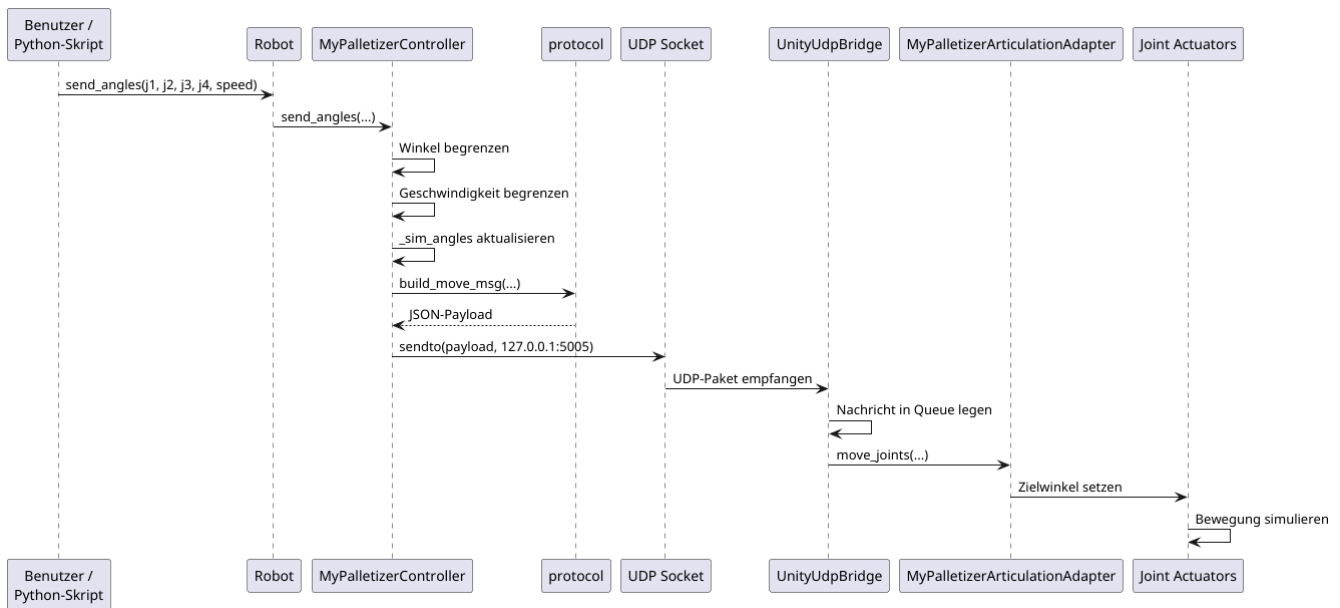
Die Kommunikation zur Unity-Simulation erfolgt über zwei getrennte UDP-Kanäle:

- Port **5005** für Roboterbefehle
- Port **5006** für Weltbefehle

Die Kommunikation mit dem realen Roboter erfolgt über `pymycobot` und eine serielle USB-Verbindung.

7.3. Szenario 1: Asynchrone Gelenkbewegung in der Simulation

Dieses Szenario beschreibt den typischen Ablauf, wenn der Benutzer einen Bewegungsbefehl im Modus `virtual` ausführt. Dabei wird ein neuer Soll-Zustand für alle vier Gelenke an die Unity-Simulation gesendet, ohne dass auf eine Rückmeldung gewartet wird.



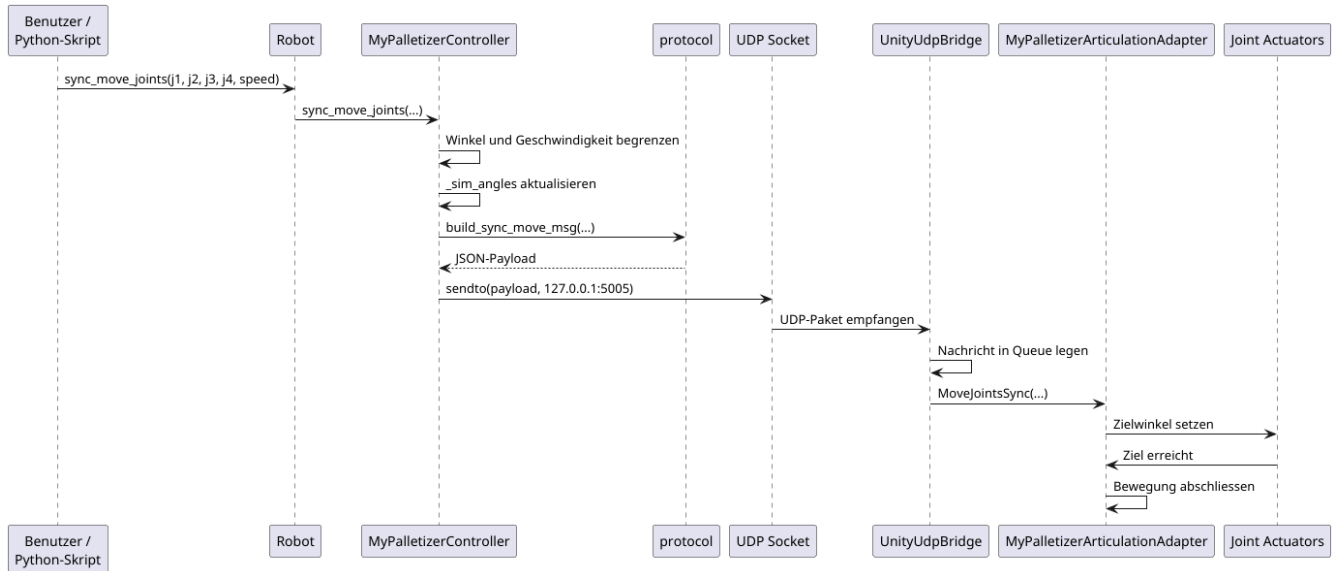
Ablauf:

1. Der Benutzer ruft `send_angles(...)` über die `Robot`-API auf.
2. `Robot` delegiert den Befehl an den `MyPalletizerController`.
3. Der Controller prüft die Werte, begrenzt Winkel und Geschwindigkeit und speichert die Zielwerte intern als Simulationszustand.
4. Anschliessend wird über das `protocol`-Modul eine JSON-Nachricht erzeugt.
5. Diese Nachricht wird per UDP an Port `5005` gesendet.
6. In Unity empfängt `UnityUdpBridge` das Paket und legt die Nachricht in eine Queue.
7. Die Verarbeitung erfolgt im Unity-Hauptthread.
8. Der Adapter steuert die simulierten Gelenke an, wodurch die Bewegung in Unity sichtbar wird.

Dieses Szenario ist der Standardfall für flüssige Bewegungen in der Simulation. Da nicht auf eine Bestätigung aus Unity gewartet wird, bleibt die Steuerung einfach und latenzarm.

7.4. Szenario 2: Synchrone Gelenkbewegung in der Simulation

Dieses Szenario beschreibt den Ablauf einer synchronen Bewegung. Dabei sollen mehrere Gelenke gemeinsam bewegt werden, und die Unity-Seite wartet intern, bis alle Gelenke ihren Zielwinkel erreicht haben.



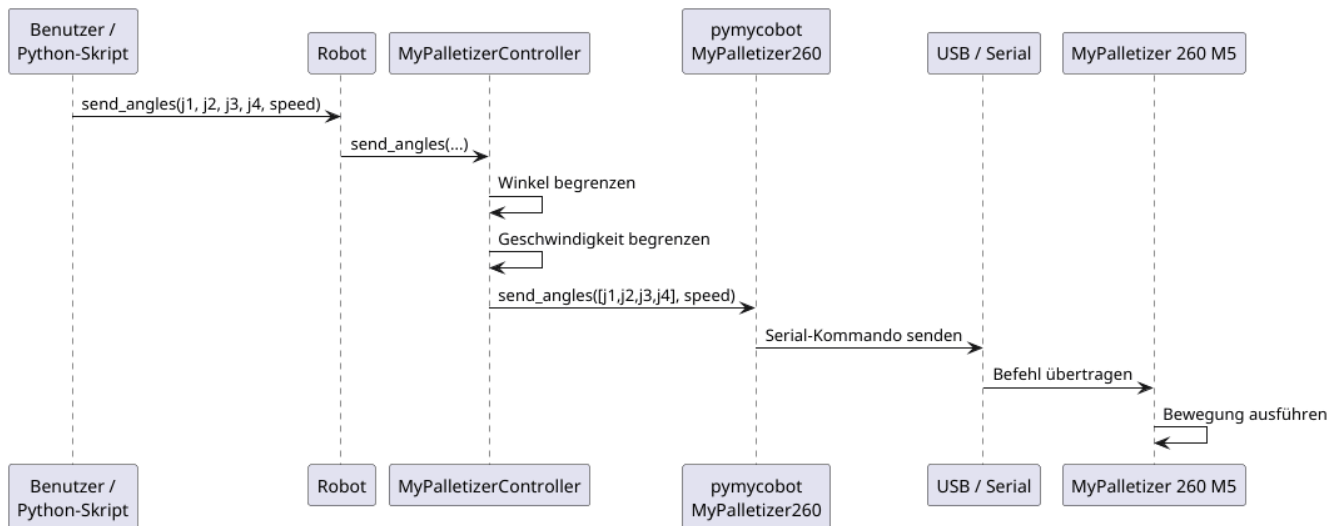
Ablauf:

1. Der Benutzer ruft `sync_move_joints(...)` über die **Robot**-API auf.
2. Der Controller validiert die Eingaben und erzeugt eine synchrone Bewegungsnachricht.
3. Die Nachricht wird über den Roboterkanal an Unity gesendet.
4. **UnityUdpBridge** verarbeitet die Nachricht im Unity-Hauptthread.
5. Der **MyPalletizerArticulationAdapter** setzt die Zielwinkel für alle Gelenke.
6. Die Adapter-Logik wartet innerhalb der Unity-Simulation, bis alle Gelenke ihren Zielwinkel erreicht haben.

Wichtig ist, dass die Synchronität hier auf Unity-Seite stattfindet. Die Python-Seite erhält aktuell keine detaillierte Rückmeldung darüber, ob die Bewegung tatsächlich abgeschlossen wurde.

7.5. Szenario 3: Bewegung auf realer Hardware

Dieses Szenario beschreibt den Ablauf im Modus **real**. Im Unterschied zur Simulation wird der Befehl nicht per UDP gesendet, sondern über die Herstellerbibliothek **pymycobot** an den realen Roboter weitergegeben.



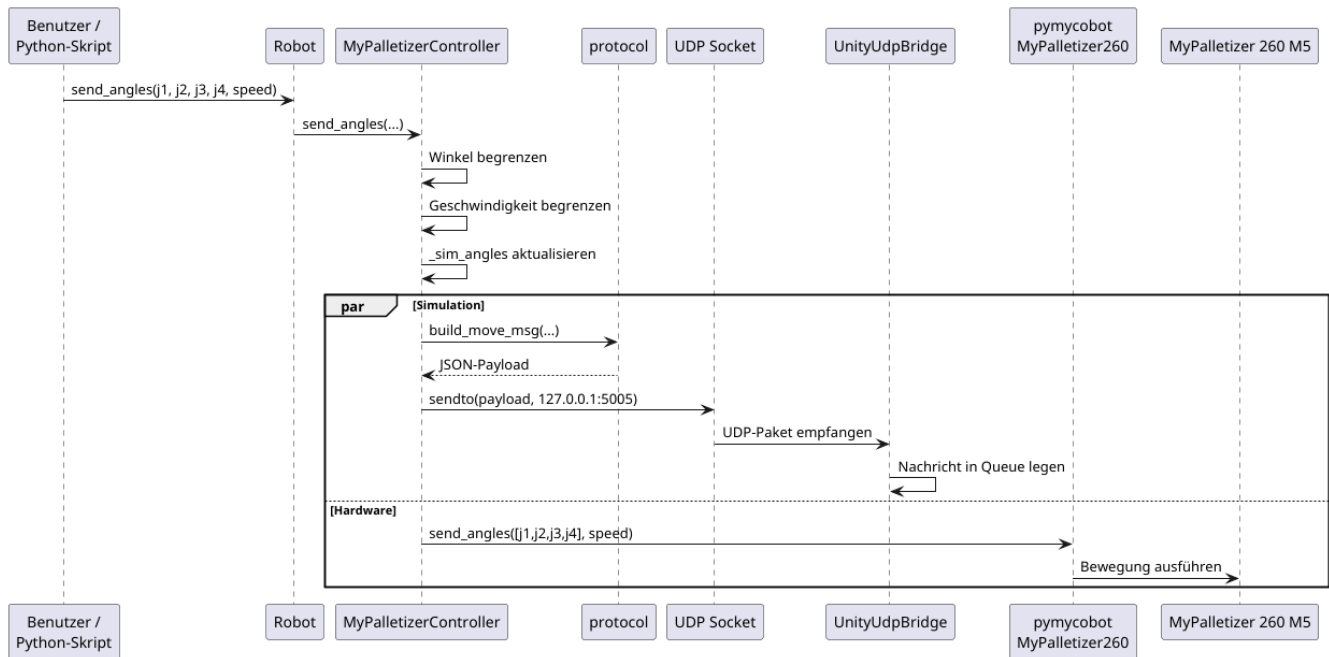
Ablauf:

1. Der Benutzer verwendet dieselbe API wie im Simulationsbetrieb.
2. **Robot** delegiert den Befehl an den **MyPalletizerController**.
3. Der Controller validiert und begrenzt die Eingaben.
4. Im Modus **real** wird der Befehl direkt an **pymycobot** weitergegeben.
5. Die Herstellerbibliothek sendet das Kommando über die serielle Verbindung an den realen Roboter.
6. Der Roboter führt die Bewegung physisch aus.

Dieses Szenario zeigt, dass dieselbe Benutzer-API auch für reale Hardware verwendet werden kann. Die technischen Details der Hardwarekommunikation bleiben vor dem Benutzer verborgen.

7.6. Szenario 4: Gleichzeitige Ausführung in Simulation und Hardware

Im Modus **both** wird derselbe Befehl sowohl an die Simulation als auch an den realen Roboter gesendet. Dies ist die eigentliche Digital-Twin-Funktion des Systems.



Ablauf:

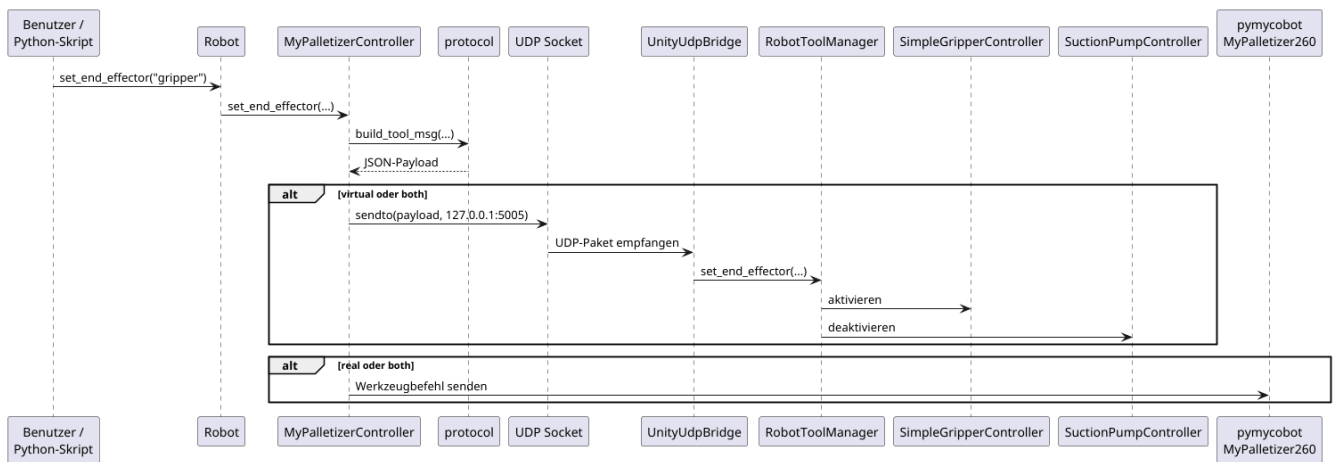
1. Der Benutzer verwendet erneut dieselbe API.
2. Der **MyPalletizerController** erkennt, dass der Modus **both** aktiv ist.
3. Der Befehl wird an die Unity-Simulation per UDP gesendet.
4. Gleichzeitig wird der Befehl über **pymycobot** an die reale Hardware weitergegeben.
5. Die Bewegung läuft dadurch in der Simulation und auf dem realen Roboter.

Dieser Modus eignet sich besonders für Demonstrationen und für den direkten Vergleich zwischen Simulation und realem Verhalten.

Dabei ist zu beachten, dass Simulation und Hardware nicht exakt synchron laufen müssen. Unterschiede können durch Laufzeiten, physikalische Effekte, serielle Kommunikation oder Modellabweichungen entstehen.

7.7. Szenario 5: Werkzeug- und End-Effektor-Steuerung

Dieses Szenario beschreibt die Steuerung von Greifer, Pumpe und aktivem Werkzeug. Diese Befehle folgen demselben Grundprinzip wie Bewegungsbefehle, werden jedoch in Unity an den **RobotToolManager** weitergegeben.



Ablauf:

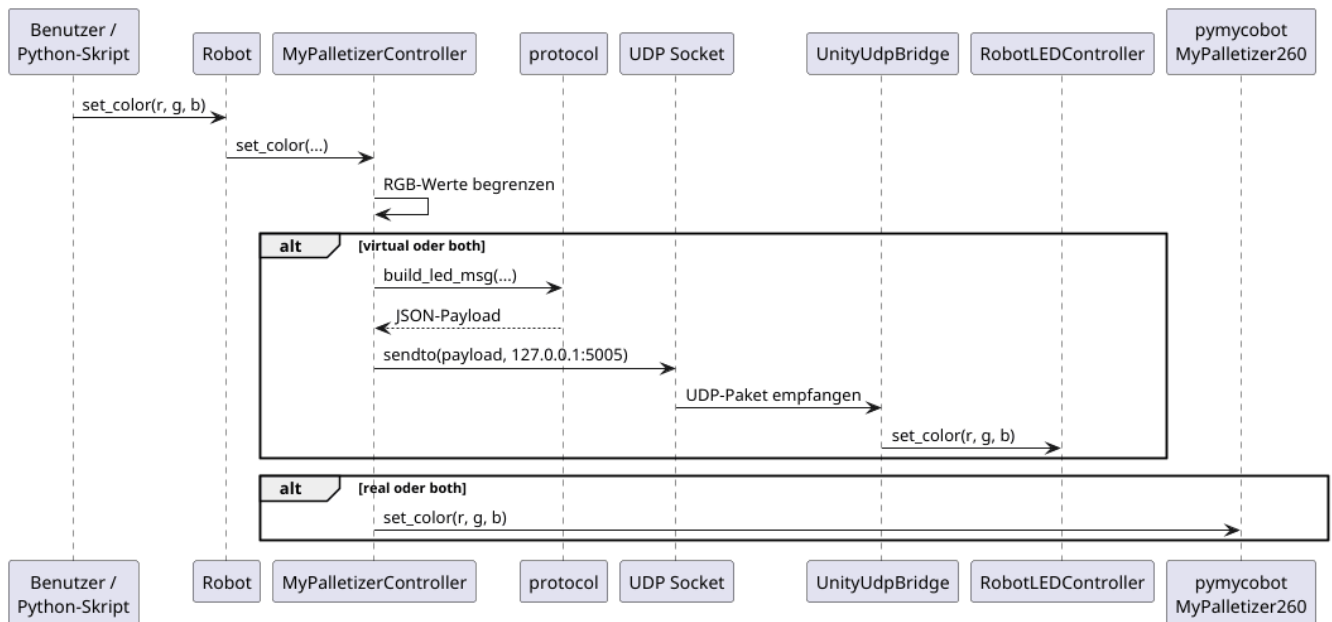
1. Der Benutzer wählt ein Werkzeug oder steuert Greifer beziehungsweise Pumpe.
2. **Robot** delegiert den Befehl an den **MyPalletizerController**.
3. Für die Simulation wird eine passende JSON-Nachricht erzeugt.
4. Unity empfängt die Nachricht über den Roboterkanal auf Port **5005**.
5. Der **RobotToolManager** aktiviert das passende Werkzeug oder leitet den Befehl an Greifer oder Pumpe weiter.
6. Im Modus **real** oder **both** wird zusätzlich der passende Hardwarebefehl über **pymycobot** ausgeführt.

Beispiele für solche Befehle sind:

- **set_end_effector("gripper")**
- **set_gripper_state(...)**
- **pump_on()**
- **pump_off()**

7.8. Szenario 6: LED-Steuerung

Die LED-Steuerung folgt ebenfalls dem Roboterkanal. RGB-Werte werden auf Python-Seite validiert und anschliessend an Simulation und/oder Hardware weitergeleitet.

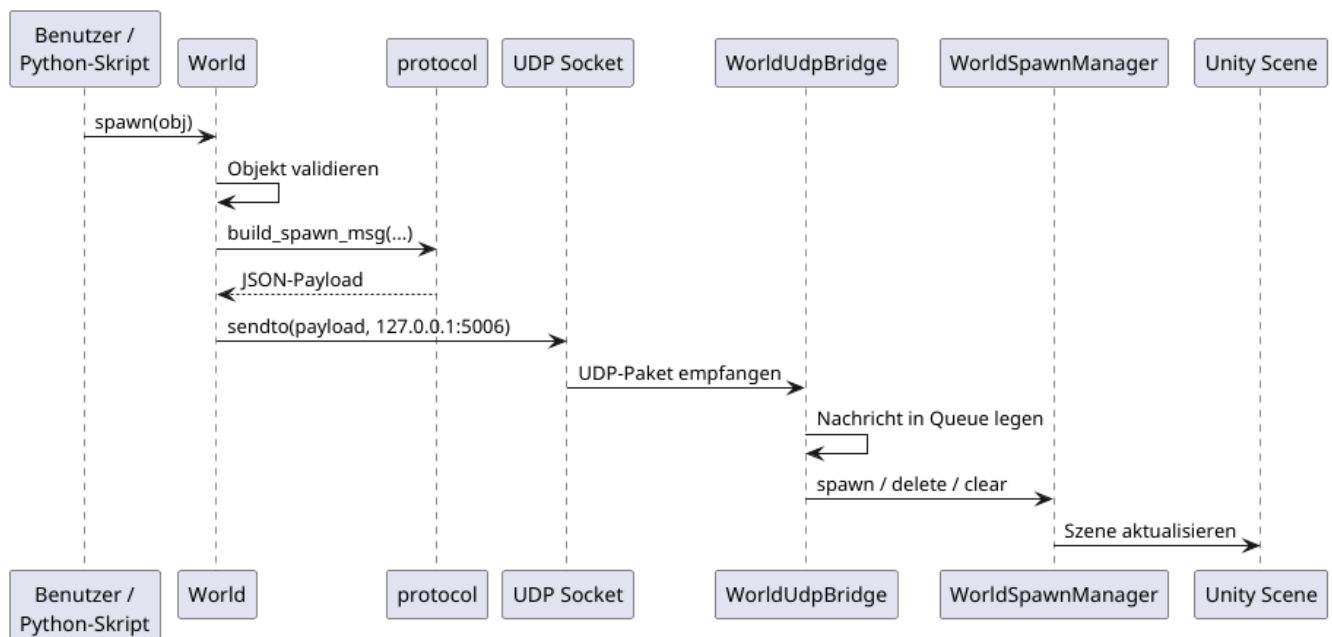


Ablauf:

1. Der Benutzer ruft `set_color(r, g, b)` auf.
2. Der Controller begrenzt die RGB-Werte auf gültige Wertebereiche.
3. Für Unity wird eine JSON-Nachricht erzeugt und über Port `5005` gesendet.
4. In Unity aktualisiert der `RobotLEDController` die visuelle LED-Darstellung.
5. Im Modus `real` oder `both` wird zusätzlich die LED des realen Roboters gesetzt.

7.9. Szenario 7: Verwaltung der Simulationswelt

Neben der Robotersteuerung unterstützt das System auch Änderungen an der Simulationswelt. Diese Befehle laufen nicht über `Robot`, sondern über die Klasse `World`.



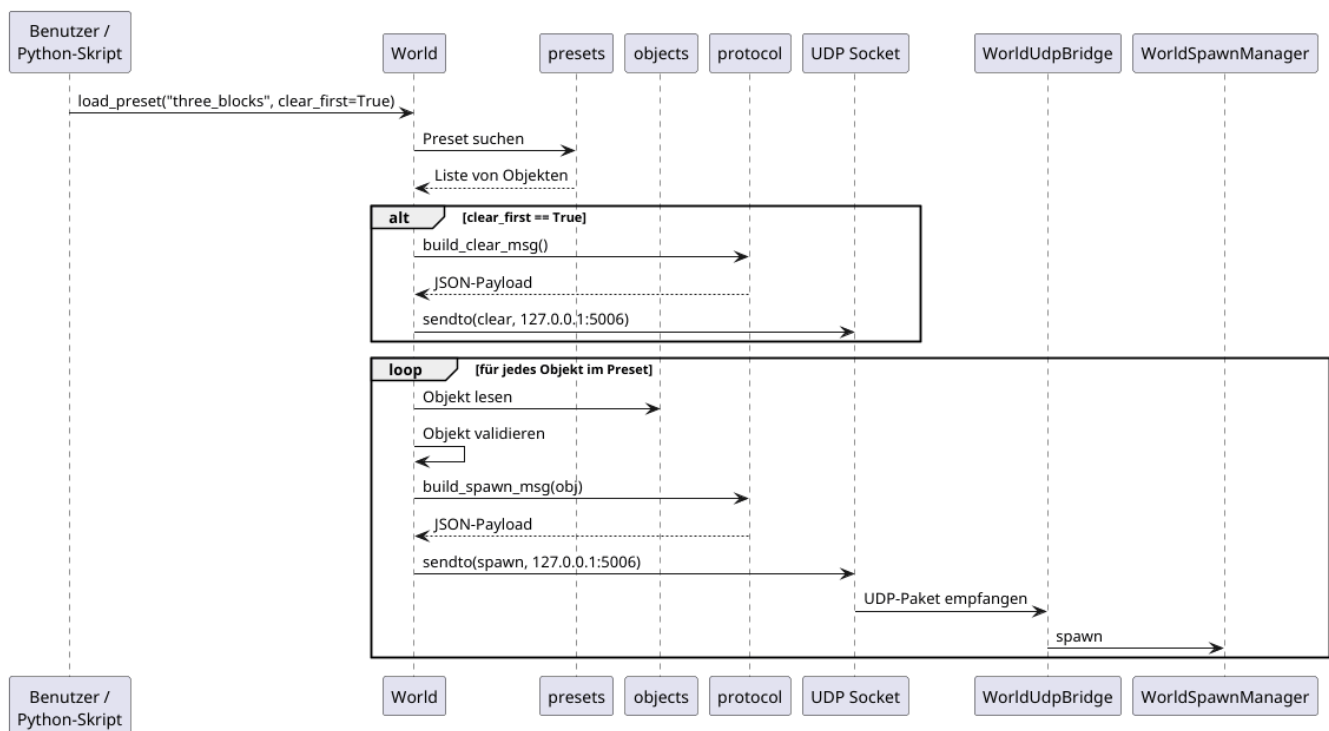
Ablauf:

1. Der Benutzer erzeugt ein Objekt und ruft `world.spawn(obj)` auf.
2. `World` prüft das Objekt und erzeugt eine passende JSON-Nachricht.
3. Die Nachricht wird per UDP an Port `5006` gesendet.
4. `WorldUdpBridge` empfängt die Nachricht und legt sie in eine Queue.
5. Die Verarbeitung erfolgt im Unity-Hauptthread.
6. Der `WorldSpawnManager` aktualisiert die Unity-Szene und erzeugt, löscht oder leert Objekte.

Die Trennung zwischen Roboter- und Weltkanal ist wichtig, damit häufige Bewegungsbefehle und seltenere Weltänderungen unabhängig voneinander verarbeitet werden können.

7.10. Szenario 8: Laden eines Presets

Presets ermöglichen das schnelle Erzeugen wiederholbarer Szenarien. Dabei werden mehrere Weltobjekte nacheinander über den Weltkanal an Unity übertragen.



Ablauf:

1. Der Benutzer ruft `load_preset(...)` auf.
2. `World` lädt das gewünschte Preset aus dem Modul `presets`.
3. Falls `clear_first` aktiv ist, wird zuerst eine `clear`-Nachricht an Unity gesendet.
4. Anschliessend werden alle Objekte des Presets nacheinander validiert und per `spawn`-Nachricht übertragen.
5. Unity erzeugt die Objekte in der Szene.

Presets dienen vor allem der Reproduzierbarkeit von Tests und Demonstrationen.

7.11. Fehler- und Grenzfälle zur Laufzeit

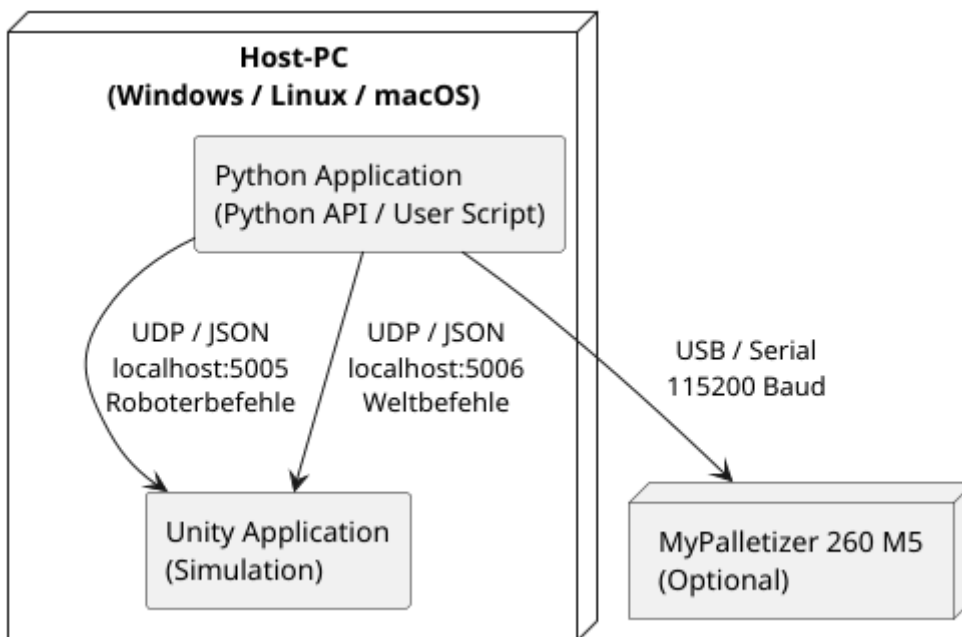
Die wichtigsten Laufzeitgrenzen ergeben sich aus Kommunikation, Hardwareverfügbarkeit und Modellgenauigkeit.

Situation	Verhalten	Auswirkung
Ungültige Gelenkwinkel	Werte werden auf definierte Grenzwerte begrenzt.	Der Roboter erhält nur zulässige Sollwerte.
Ungültige Geschwindigkeit	Die Geschwindigkeit wird auf den erlaubten Bereich begrenzt.	Zu schnelle oder ungültige Bewegungen werden vermieden.
Ungültige RGB-Werte	RGB-Werte werden auf den Bereich 0 bis 255 begrenzt.	Die LED-Steuerung bleibt gültig.
UDP-Paketverlust	Es erfolgt keine automatische Wiederholung durch UDP.	Bei zustandsbasierten Bewegungsbefehlen kann ein späteres Paket den Zielzustand wieder korrigieren.
Unity nicht gestartet	UDP-Nachrichten können gesendet werden, werden aber nicht verarbeitet.	In der Simulation ist keine Auswirkung sichtbar.
Hardware nicht verbunden	Hardwarebefehle können im Modus real oder both fehlschlagen.	Der Benutzer muss Verbindung, Port und Stromversorgung prüfen.
Unterschiedliche Laufzeiten in both	Simulation und Hardware reagieren nicht zwingend exakt gleichzeitig.	Kleine Abweichungen zwischen virtuellem und realem Verhalten sind möglich.
Ungültiges Weltobjekt	World validiert Objektparameter vor dem Versand.	Fehlerhafte Objekte werden nicht oder nur nach Korrektur übertragen.

8. Deployment View

Die Verteilungssicht beschreibt die physische Infrastruktur und die Softwareumgebung, in der das System betrieben wird. Sie zeigt, auf welchen Laufzeitumgebungen die einzelnen Komponenten ausgeführt werden und über welche Verbindungen sie miteinander kommunizieren.

Das System ist für den Betrieb auf einem normalen Rechner ausgelegt. Die Python-Anwendung und die Unity-Simulation laufen in der Regel auf demselben Host-PC. Optional kann zusätzlich der reale MyPalletizer 260 M5 über USB angeschlossen werden.



8.1. Host-PC

Der Host-PC ist die zentrale Laufzeitumgebung des Systems. Auf ihm werden die Python-Steuerung und die Unity-Simulation ausgeführt.

Der Host-PC benötigt folgende Softwarekomponenten:

- **Python Runtime ab Version 3.12** Erforderlich für die Ausführung der Steuerlogik, der Python-API und der Benutzerskripte.
- **Python-Abhängigkeiten** Benötigt werden die projektinternen Module sowie externe Abhängigkeiten wie `pymycobot`, sofern die reale Hardware verwendet wird.
- **Unity-Anwendung** Enthält die Simulation, die Visualisierung und die Empfängerlogik für die UDP-Kommunikation.
- **USB-Seriell-Verbindung** Wird benötigt, wenn der reale Roboter im Modus `real` oder `both` verwendet werden soll.
- **Freie UDP-Ports** Die Ports `5005` und `5006` müssen verfügbar sein, damit Python mit der Unity-Simulation kommunizieren kann.

8.2. Verteilung der Komponenten

Komponente	Laufzeitumgebung	Beschreibung
Python Application	Host-PC	Führt die Benutzerskripte und die Python-API aus. Sie entscheidet anhand des Betriebsmodus, ob Befehle an Unity, an die reale Hardware oder an beide Zielsysteme weitergegeben werden.
Unity Application	Host-PC	Führt die Simulation aus. Sie empfängt UDP-Nachrichten, visualisiert den Roboter und verwaltet die Simulationswelt.
MyPalletizer 260 M5	Externes Gerät	Optionaler realer Roboter. Er wird über USB/Serial angebunden und nur in den Modi real und both benötigt.

8.3. Kommunikation

Die Kommunikation zwischen den Komponenten erfolgt über einfache, lokal verfügbare Schnittstellen.

Verbindung	Technologie	Zweck
Python zu Unity: Roboterkanal	UDP / JSON, Port 5005	Überträgt Roboterbefehle wie Gelenkbewegungen, Werkzeugbefehle, Greifer-, Pumpen- und LED-Steuerung.
Python zu Unity: Weltkanal	UDP / JSON, Port 5006	Überträgt Weltbefehle wie spawn , delete , clear und das Laden von Szenarien.
Python zu realem Roboter	USB / Serial, 115200 Baud	Überträgt Hardwarebefehle über pymycobot an den MyPalletizer 260 M5.

Die Kommunikation zwischen Python und Unity erfolgt standardmässig über **localhost**. Dadurch müssen keine zusätzlichen Netzwerkdienste betrieben werden. Falls Python und Unity auf unterschiedlichen Rechnern ausgeführt werden sollen, müssen IP-Adresse, Firewall-Regeln und erreichbare UDP-Ports entsprechend angepasst werden.

8.4. Bereitstellung

Die Software kann in zwei Varianten bereitgestellt werden:

- **Fertige Anwendung** Die Unity-Simulation wird als ausführbares Artefakt für das jeweilige Betriebssystem bereitgestellt. Der Benutzer kann die passende Version herunterladen und starten, ohne die Simulation selbst in Unity bauen zu müssen.
- **Entwicklungsumgebung** Für Anpassungen oder Erweiterungen kann das Unity-Projekt direkt geöffnet werden. Dadurch können Modelle, Szenen, Skripte oder Simulationsverhalten verändert werden.

Falls eine CI/CD-Pipeline verwendet wird, kann der Build-Prozess automatisiert werden. Dabei werden für die unterstützten Betriebssysteme jeweils passende Artefakte erstellt. Dies vereinfacht die Nutzung erheblich, insbesondere für Anwender ohne Unity-Vorkenntnisse.

8.5. MyPalletizer 260 M5

Der reale Roboter ist ein optionaler Bestandteil der Laufzeitumgebung. Er wird benötigt, wenn das System im Modus **real** oder **both** betrieben wird.

Für den Betrieb sind insbesondere folgende Punkte relevant:

- Verbindung über USB/Serial
- korrekter serieller Port auf dem Host-PC
- Baudrate **115200**
- passende Firmware auf dem Roboter
- installierte Python-Bibliothek **pymycobot**
- externe Stromversorgung für einen zuverlässigen Betrieb

Ohne angeschlossenen Roboter kann das System weiterhin im Modus **virtual** verwendet werden.

8.6. Betriebsarten

Das System kann in drei Varianten betrieben werden:

Modus	Benötigte Komponenten	Beschreibung
virtual	Python Application, Unity Application	Befehle werden nur an die Unity-Simulation gesendet. Dieser Modus eignet sich für Entwicklung, Tests und Demonstrationen ohne reale Hardware.
real	Python Application, MyPalletizer 260 M5	Befehle werden nur an den realen Roboter gesendet. Unity muss für diesen Modus nicht zwingend laufen.
both	Python Application, Unity Application, MyPalletizer 260 M5	Befehle werden sowohl an die Simulation als auch an den realen Roboter gesendet. Dieser Modus eignet sich für Demonstrationen und den Vergleich zwischen virtuellem und realem Verhalten.

8.7. Typische Deployment-Szenarien

8.7.1. Nur Simulation

In diesem Szenario laufen Python und Unity auf demselben Host-PC. Der reale Roboter ist nicht angeschlossen.

Diese Variante ist besonders geeignet für:

- Entwicklung von Bewegungsabläufen
- Übungen ohne Hardwarezugriff
- Tests von Weltobjekten und Presets
- Demonstrationen der Simulation

8.7.2. Simulation und reale Hardware

In diesem Szenario laufen Python und Unity auf dem Host-PC, zusätzlich ist der MyPalletizer über USB angeschlossen.

Diese Variante ist geeignet für:

- Vergleich zwischen Simulation und Hardware
- Demonstrationen des Digital Twins
- Validierung von Bewegungsabläufen auf realer Hardware

8.7.3. Nur reale Hardware

In diesem Szenario läuft nur die Python-Anwendung auf dem Host-PC. Unity wird nicht benötigt.

Diese Variante ist geeignet für:

- direkte Hardwaretests
- Ausführung bereits validierter Bewegungsabläufe
- Nutzung der Python-API als vereinfachte Hardware-Schnittstelle

8.8. Deployment-Randbedingungen

Für einen stabilen Betrieb sind folgende Randbedingungen zu beachten:

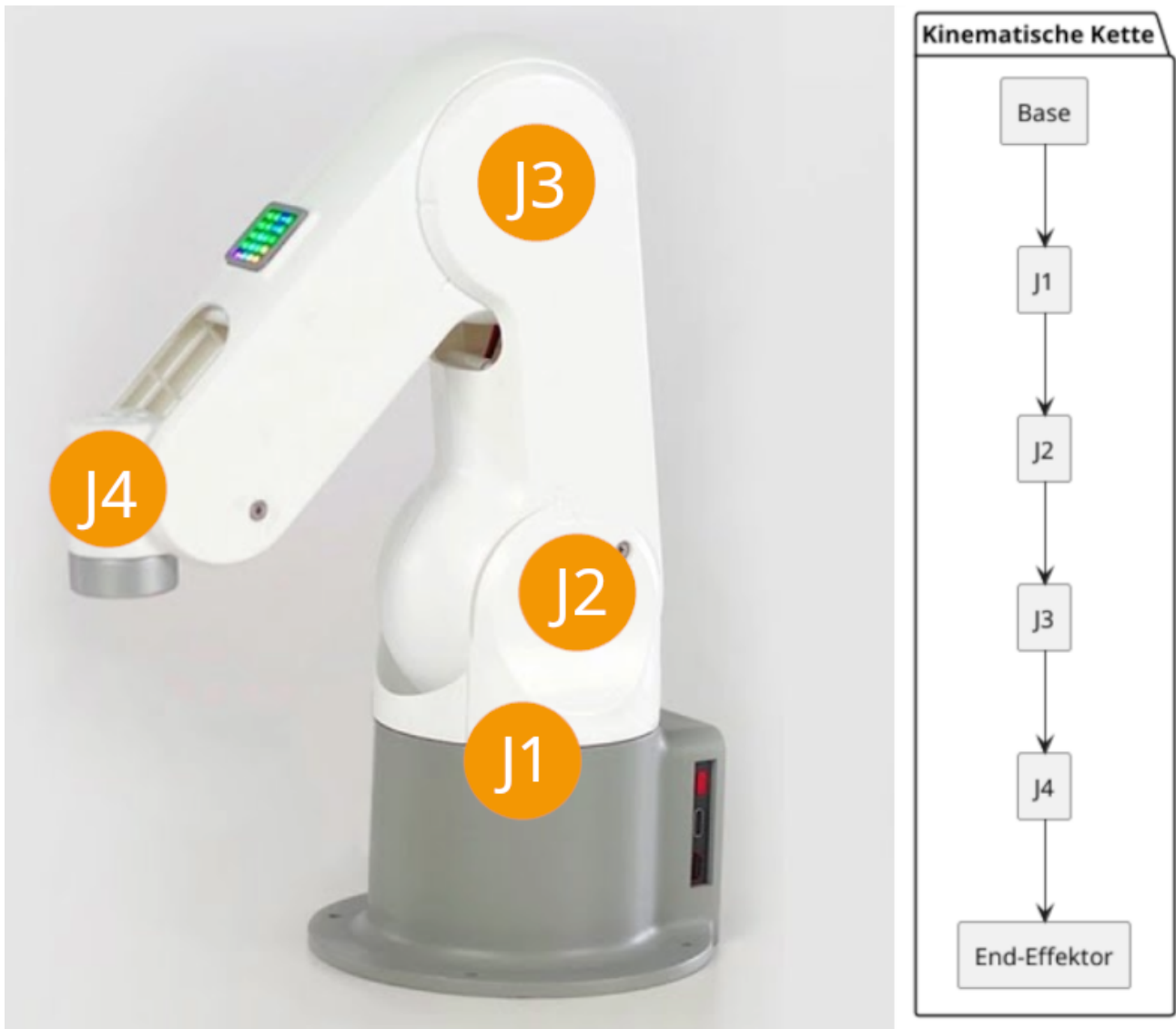
- Die UDP-Ports **5005** und **5006** dürfen nicht durch andere Anwendungen belegt sein.
- Firewall-Regeln dürfen die lokale UDP-Kommunikation nicht blockieren.
- Der serielle Port des Roboters muss korrekt angegeben werden.
- Der reale Roboter muss eingeschaltet und korrekt verbunden sein.
- Unity muss gestartet sein, bevor Befehle im Modus **virtual** oder **both** sichtbar verarbeitet werden können.
- Bei Betrieb auf unterschiedlichen Rechnern müssen IP-Adressen und Portfreigaben angepasst werden.

9. Cross-cutting Concepts

In diesem Kapitel werden Konzepte beschrieben, die in mehreren Teilen des Systems verwendet werden. Diese Konzepte betreffen nicht nur einzelne Bausteine, sondern prägen die Architektur über mehrere Schichten hinweg.

9.1. Simulationskonzept

Die Simulation des Roboters wird in Unity umgesetzt. Der Roboter besteht aus mehreren miteinander verbundenen Gelenken. Dadurch bewegen sich untergeordnete Bauteile mit, wenn ein übergeordnetes Gelenk bewegt wird. Beispielsweise bewegt sich ein nachgelagertes Gelenk wie **J3** mit, wenn ein vorgelagertes Gelenk wie **J2** gedreht wird.



Die Trennung in einzelne Gelenke ermöglicht es, Bewegungsbefehle aus der Python-Anwendung direkt auf die simulierte Roboterstruktur abzubilden. Zielwinkel für die einzelnen Achsen können unabhängig verarbeitet und in der Simulation dargestellt werden.

Die Simulation verwendet dabei ein vereinfachtes Modell des realen Roboters. Sie soll ein plausibles und nachvollziehbares Verhalten zeigen, ist jedoch keine vollständig physikalisch exakte Nachbildung. Abweichungen können insbesondere durch vereinfachte Modelle, CAD-

Abweichungen oder Unterschiede in der physikalischen Simulation entstehen.

9.2. Kommunikationskonzept

Die Kommunikation zwischen Python und Unity basiert auf UDP und JSON. Python erzeugt strukturierte JSON-Nachrichten und sendet diese an die Unity-Simulation.

Dabei werden zwei Kommunikationskanäle unterschieden:

Kanal	Port	Zweck
Roboterkanal	5005	Überträgt Roboterbefehle wie Gelenkbewegungen, Werkzeugsteuerung, Pumpensteuerung und LED-Befehle.
Weltkanal	5006	Überträgt Weltbefehle wie <code>spawn</code> , <code>delete</code> , <code>clear</code> und das Laden von Szenarien.

Die Trennung der Kanäle sorgt dafür, dass häufige Roboterbefehle nicht mit selteneren Weltbefehlen vermischt werden. Dadurch bleibt die Kommunikation übersichtlicher und leichter wartbar.

Die Nachrichten enthalten immer eine Protokollversion `v` und ein Feld `type`. Dadurch kann Unity erkennen, welche Art von Befehl verarbeitet werden soll.

9.3. Betriebsartenkonzept

Das System unterstützt drei Betriebsarten:

Modus	Beschreibung
<code>virtual</code>	Befehle werden nur an die Unity-Simulation gesendet. Dieser Modus eignet sich für Entwicklung und Tests ohne reale Hardware.
<code>real</code>	Befehle werden nur an den realen MyPalletizer 260 M5 gesendet. Die Kommunikation erfolgt über <code>pymycobot</code> und USB/Serial.
<code>both</code>	Befehle werden sowohl an die Simulation als auch an die reale Hardware gesendet. Dieser Modus eignet sich für Demonstrationen und den Vergleich zwischen simuliertem und realem Verhalten.

Der Benutzer arbeitet in allen Betriebsarten mit derselben Python-API. Die Entscheidung, wohin ein Befehl gesendet wird, erfolgt intern im `MyPalletizerController`.

9.4. Zustandskonzept

Im virtuellen Betrieb speichert die Python-Seite den zuletzt gesendeten Soll-Zustand der Gelenke. Dieser Zustand wird in `_sim_angles` verwaltet und dient dazu, auch ohne reale Hardware sinnvolle

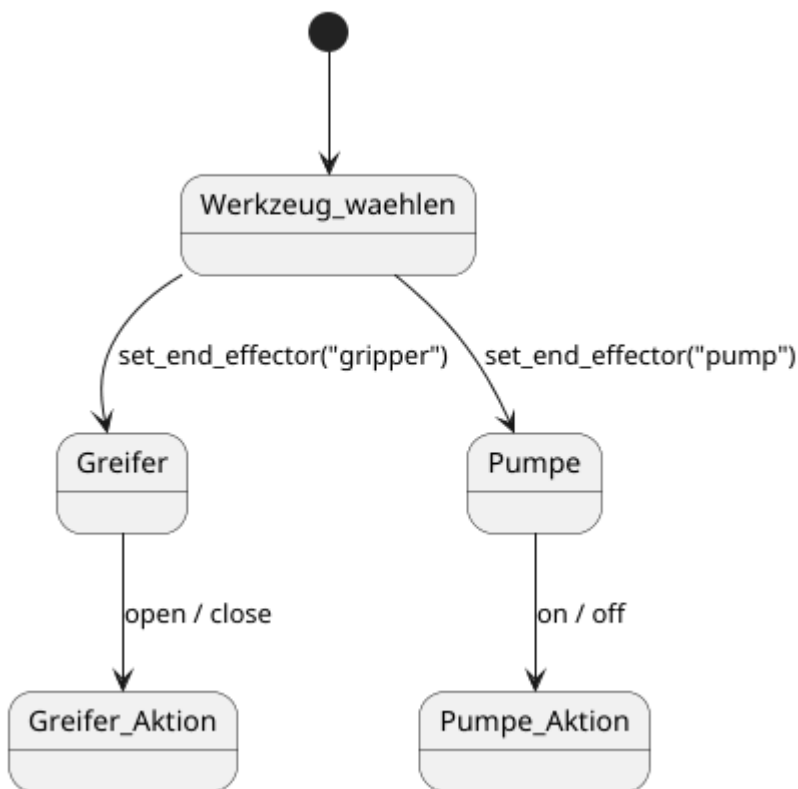
Werte für `get_angles()` zurückgeben zu können.

Dabei handelt es sich nicht um einen gemessenen Ist-Zustand aus Unity, sondern um den zuletzt gesetzten Zielzustand. Das ist wichtig, da aktuell kein vollständiger Rückkanal aus der Simulation existiert.

Für die reale Hardware kann der Zustand über `pymycobot` vom Roboter abgefragt werden, sofern die entsprechende Funktion verfügbar ist. Im Modus `both` können Simulation und Hardware zeitweise voneinander abweichen, weil sie nicht über einen gemeinsamen Rückkanal synchronisiert werden.

9.5. End-Effektor-Konzept

Der Roboter unterstützt unterschiedliche Werkzeuge, insbesondere Greifer und Pumpe. Diese Werkzeuge können über die Python-API ausgewählt und in der Simulation nachgebildet werden.



Dieses Konzept sorgt dafür, dass verschiedene Werkzeugarten über dieselbe Schnittstelle angesprochen werden können. Der Benutzer muss nicht wissen, welche Unity-Komponente oder Hardwarefunktion intern verwendet wird.

In der Simulation werden Werkzeugbefehle durch spezialisierte Komponenten verarbeitet:

- `RobotToolManager` koordiniert die Werkzeugauswahl.
- `SimpleGripperController` bildet das Verhalten des Greifers nach.
- `SuctionPumpController` bildet das Verhalten der Vakuumpumpe nach.

9.6. Objekt- und Szenarienkonzept

Für die Simulationswelt werden einfache Objekte wie Boxen, Kugeln oder Zylinder verwendet. Diese können über die Python-Seite erstellt und in Unity dargestellt werden.

Objekte enthalten typischerweise folgende Eigenschaften:

- Name
- Objekttyp
- Position
- Rotation
- Skalierung
- Farbe

Objekte werden über ihren Namen angesprochen. So können sie später gezielt gelöscht oder verändert werden.

Zusätzlich unterstützt das System vordefinierte Szenarien, sogenannte **Presets**. Diese ermöglichen einen einfacheren Einstieg in die Nutzung und sorgen für reproduzierbare Testszenarien. Beispiele für Presets sind `empty`, `three_blocks`, `pick_and_place_basic` und `sorting_task`.

9.7. Threading-Konzept in Unity

UDP-Nachrichten werden in Unity nicht direkt auf Unity-Objekte angewendet. Stattdessen werden eingehende Nachrichten zunächst empfangen und in eine Queue gelegt.

Die eigentliche Verarbeitung erfolgt anschliessend im Unity-Hauptthread. Dieses Vorgehen ist wichtig, da Unity-Objekte nicht sicher aus beliebigen Hintergrund-Threads verändert werden sollten.

Dieses Konzept betrifft insbesondere:

- `UnityUdpBridge`
- `WorldUdpBridge`
- `MyPalletizerArticulationAdapter`
- `WorldSpawnManager`

Dadurch bleibt die Verarbeitung stabiler und Threading-Probleme werden vermieden.

9.8. Fehlerbehandlung und Validierung

Ein weiteres wichtiges Konzept ist die Validierung von Eingaben vor der Ausführung.

Dazu gehören unter anderem:

- Begrenzung von Gelenkwinkeln

- Begrenzung der Geschwindigkeit
- Begrenzung von RGB-Werten
- Prüfung von Objektparametern
- Prüfung der Betriebsart
- Fehlerbehandlung bei fehlender oder ungültiger Konfiguration

Dadurch werden fehlerhafte Befehle möglichst früh erkannt und unzulässige Zustände vermieden. Beispielsweise sollte im Modus **real** oder **both** ein gültiger serieller Port angegeben sein, da sonst keine Verbindung zur Hardware aufgebaut werden kann.

9.9. Testkonzept

Das Testkonzept umfasst sowohl Unit-Tests für die Python-API als auch Tests für Unity-Komponenten. Dabei werden typische Anwendungsfälle, Grenzfälle und Fehlerfälle abgedeckt, um die Stabilität und Zuverlässigkeit des Systems sicherzustellen.

Zusätzlich wurden Systemtests definiert und manuell ausgeführt. Diese Tests decken die Interaktion zwischen Python und Unity ab und prüfen, ob die Kommunikation korrekt funktioniert und die Simulation wie erwartet reagiert.

Die Systemtests wurden im GitHub-Wiki dokumentiert, damit sie auch von anderen Benutzern nachvollzogen werden können:

[Systemtests im GitHub-Wiki](#)

9.10. Sicherheitskonzept

Die Simulation dient auch als Sicherheitsmechanismus. Bewegungsabläufe können zunächst virtuell getestet werden, bevor sie auf dem realen Roboter ausgeführt werden. Dadurch sinkt das Risiko, die Hardware durch falsche Winkel, ungeeignete Bewegungsabläufe oder fehlerhafte Skripte zu beschädigen.

Die Sicherheit wird zusätzlich durch Validierungen unterstützt:

- Gelenkwinkel werden begrenzt.
- Geschwindigkeiten werden begrenzt.
- Ungültige Objektparameter werden erkannt.
- Die Hardwareansteuerung ist gekapselt.

Trotzdem ersetzt die Simulation keine vollständige Sicherheitsprüfung am realen Roboter. Vor der Ausführung auf der Hardware muss weiterhin geprüft werden, ob der Arbeitsbereich frei ist und keine Kollisionen auftreten können.

10. Architecture Decisions

Die folgenden Architectural Decision Records (ADR) beschreiben wichtige Architekturentscheidungen des Projekts. Sie zeigen, welche Alternativen betrachtet wurden, welche Lösung gewählt wurde und welche Konsequenzen daraus entstehen.

10.1. ADR 1: Einheitliche Python-API für Simulation und Hardware

- **Kontext:** Der Benutzer soll nicht zwischen verschiedenen APIs für Simulation und reale Hardware unterscheiden müssen.
- **Entscheidung:** Die Klassen `Robot` und `World` bilden die zentrale Benutzerschnittstelle. Intern wird die eigentliche Ausführung an den `MyPalletizerController` oder an die Weltkommunikation delegiert.
- **Vorteile gegenüber getrennten APIs:**
 - Der Benutzer muss nur eine einheitliche Schnittstelle lernen.
 - Beispielcode kann einfacher zwischen Simulation und realem Roboter wiederverwendet werden.
 - Die Bedienung wird konsistenter, da dieselben Methoden unabhängig vom Ausführungsmodus verwendet werden können.
 - Änderungen an der internen Kommunikation wirken sich weniger stark auf den Benutzercode aus.
- **Nachteile gegenüber getrennten APIs:**
 - Die Python-API muss mehr Logik übernehmen, zum Beispiel Moduswahl, Validierung und Weiterleitung.
 - Unterschiede zwischen Simulation und echter Hardware können durch die einheitliche API teilweise verdeckt werden.
 - Fehler können schwieriger zu lokalisieren sein, da zusätzliche Abstraktionsschichten beteiligt sind.
- **Konsequenz:** Die Benutzung des Systems wird einfacher und konsistenter. Gleichzeitig übernimmt die Python-Seite mehr Verantwortung, da sie Validierung, Moduswahl und Weiterleitung der Befehle kapselt.

10.2. ADR 2: Kapselung der Herstellerbibliothek `pymycobot`

- **Kontext:** Der reale Roboter soll unterstützt werden, ohne dass der Benutzer direkt von der Herstellerbibliothek abhängig ist.
- **Entscheidung:** Die Bibliothek `pymycobot` wird nur intern im `MyPalletizerController` verwendet. Der Benutzer arbeitet ausschliesslich mit der eigenen `Robot`-Schnittstelle.
- **Vorteile gegenüber direkter Nutzung von `pymycobot`:**

- Die öffentliche API bleibt stabiler, auch wenn sich die Herstellerbibliothek ändert.
- Der Benutzercode wird unabhängiger von herstellerspezifischen Details.
- Hardware-spezifische Fehlerbehandlung und Validierung können zentral umgesetzt werden.
- Ein späterer Wechsel oder die Erweiterung auf andere Roboter wird einfacher.
- **Nachteile gegenüber direkter Nutzung von `pymycobot`:**
 - Nicht alle Funktionen der Herstellerbibliothek sind automatisch verfügbar.
 - Die eigene Abstraktionsschicht muss gepflegt und getestet werden.
 - Neue Funktionen aus `pymycobot` müssen manuell in die eigene API übernommen werden.
 - Bei Fehlern muss teilweise geprüft werden, ob sie in der eigenen Schicht oder in `pymycobot` entstehen.
- **Konsequenz:** Externe Abhängigkeiten bleiben gekapselt und die API nach aussen bleibt stabiler. Dafür entsteht eine zusätzliche Abstraktionsschicht, die gepflegt werden muss.

10.3. ADR 3: Trennung von Simulation und realer Hardware

- **Kontext:** Dasselbe System soll sowohl mit der Simulation als auch mit dem realen Roboter funktionieren.
- **Entscheidung:** Die Simulation wird über UDP angesprochen, die reale Hardware über `pymycobot` und die serielle Verbindung. Die Auswahl erfolgt intern über die Modi `virtual`, `real` und `both`.
- **Vorteile gegenüber einer gemeinsamen Ausführungsschicht:**
 - Die technischen Unterschiede zwischen Simulation und Hardware können sauber behandelt werden.
 - Die Simulation bleibt unabhängig von der seriellen Roboterkommunikation.
 - Der reale Roboter kann über die vom Hersteller vorgesehene Bibliothek angesteuert werden.
 - Der Modus `both` ermöglicht den parallelen Vergleich zwischen Simulation und echter Bewegung.
- **Nachteile gegenüber einer gemeinsamen Ausführungsschicht:**
 - Es müssen mehrere Kommunikationswege gepflegt werden.
 - Das Verhalten von Simulation und realem Roboter kann voneinander abweichen.
 - Tests müssen sicherstellen, dass alle Modi korrekt funktionieren.
 - Die interne Architektur wird komplexer, weil Befehle je nach Modus unterschiedlich weitergeleitet werden.
- **Konsequenz:** Dieselbe Python-API kann für Simulation, reale Hardware oder beide gleichzeitig verwendet werden. Dadurch bleibt die Benutzerschnittstelle einfach, obwohl intern unterschiedliche Kommunikationswege verwendet werden.

10.4. ADR 4: Trennung von Roboter- und Weltkommunikation

- **Kontext:** Das System verarbeitet zwei unterschiedliche Arten von Befehlen: Roboterbefehle und Befehle zur Verwaltung der Simulationswelt.
- **Entscheidung:** Diese beiden Bereiche werden über getrennte UDP-Kanäle umgesetzt. Port `5005` wird für die Robotersteuerung verwendet, Port `5006` für Weltbefehle.
- **Vorteile gegenüber einem gemeinsamen UDP-Kanal:**
 - Roboterbefehle und Weltbefehle sind klar voneinander getrennt.
 - Die Verarbeitung auf Unity-Seite kann einfacher strukturiert werden.
 - Fehler in der Weltverwaltung beeinflussen die Robotersteuerung weniger direkt.
 - Die Kommunikation ist leichter zu erweitern, wenn später weitere Weltbefehle hinzukommen.
- **Nachteile gegenüber einem gemeinsamen UDP-Kanal:**
 - Es müssen zwei Ports konfiguriert und dokumentiert werden.
 - Firewall- oder Netzwerkeinstellungen können etwas aufwendiger werden.
 - Die Python-Seite muss wissen, welcher Befehl über welchen Kanal gesendet wird.
 - Bei der Fehlersuche müssen zwei Kommunikationswege berücksichtigt werden.
- **Konsequenz:** Die Kommunikation bleibt klar getrennt. Häufige Bewegungsbefehle des Roboters werden nicht mit Weltbefehlen wie `spawn`, `delete` oder `clear` vermischt.

10.5. ADR 5: Verwendung von Presets für wiederholbare Szenarien

- **Kontext:** Typische Testszenarien sollen schnell geladen und reproduzierbar ausgeführt werden können.
- **Entscheidung:** Vordefinierte Szenarien werden als Presets auf Python-Seite bereitgestellt und über `World.load_preset(...)` geladen.
- **Vorteile gegenüber manueller Szenenerstellung:**
 - Testszenarien können schnell und wiederholbar aufgebaut werden.
 - Demonstrationen werden zuverlässiger, da immer dieselbe Ausgangssituation geladen wird.
 - Benutzer müssen häufig benötigte Objekte nicht jedes Mal manuell erstellen.
 - Presets eignen sich gut für Unterrichtsbeispiele und wiederholbare Tests.
- **Nachteile gegenüber manueller Szenenerstellung:**
 - Presets müssen gepflegt werden, wenn sich Objekte oder Szenenlogik ändern.
 - Die Flexibilität ist geringer als bei komplett frei definierten Szenarien.
 - Zu viele Presets können unübersichtlich werden.

- Benutzer könnten sich zu stark auf vordefinierte Szenarien verlassen und Sonderfälle weniger testen.
- **Konsequenz:** Tests und Demonstrationen werden einfacher und konsistenter. Gleichzeitig müssen Presets gepflegt werden, wenn sich die Simulationswelt ändert.

10.6. ADR 6: Verwendung von Articulation Bodies in Unity

- **Kontext:** Der Roboterarm besteht aus mehreren Gelenken, die in der Simulation stabil und nachvollziehbar bewegt werden müssen.
- **Entscheidung:** Für die Simulation der Gelenkkette werden in Unity `ArticulationBody`-Komponenten verwendet.
- **Vorteile gegenüber einfachen `Transform`-Animationen oder `Rigidbody`-Konstruktionen:**
 - `ArticulationBody` ist besser für zusammenhängende Gelenkstrukturen geeignet.
 - Gelenkbewegungen können physikalisch nachvollziehbarer simuliert werden.
 - Die Stabilität bei mehrgliedrigen Roboterarmen ist in der Regel besser als bei vielen einzelnen `Rigidbody`-Objekten.
 - Einschränkungen wie Rotationsachsen und Gelenkgrenzen lassen sich strukturierter abbilden.
- **Nachteile gegenüber einfachen `Transform`-Animationen oder `Rigidbody`-Konstruktionen:**
 - Die Einrichtung in Unity ist komplexer.
 - Die Konfiguration der Gelenke, Achsen und Limits erfordert mehr Sorgfalt.
 - Für rein visuelle Bewegungen wäre eine einfache `Transform`-Animation leichter umzusetzen.
 - Fehlerhafte Parameter können zu unerwartetem physikalischem Verhalten führen.
- **Konsequenz:** Die Abbildung des Roboterarms in Unity wird robuster und besser für zusammenhängende Gelenkstrukturen geeignet. Gleichzeitig wird die Konfiguration in Unity aufwendiger.

10.7. ADR 7: Umgang mit fehlerhaften CAD-Modellen des Herstellers

- **Kontext:** Für die Simulation wurden CAD-Modelle des Herstellers verwendet. Dabei zeigte sich, dass diese Modelle nicht vollständig mit dem realen MyPalletizer 260 M5 übereinstimmen. Dadurch kann der simulierte Roboter geometrisch vom echten Roboter abweichen.
- **Entscheidung:** Die vorhandenen CAD-Modelle werden trotzdem als Grundlage für die visuelle Simulation verwendet. Abweichungen werden dokumentiert und bei Bedarf manuell in Unity korrigiert oder durch angepasste Komponenten kompensiert.
- **Vorteile gegenüber einer vollständigen Eigenmodellierung:**
 - Die Simulation kann schneller aufgebaut werden.

- Es muss kein vollständiges 3D-Modell des Roboters von Grund auf erstellt werden.
- Die Herstellerdaten bieten trotz Abweichungen eine brauchbare visuelle Grundlage.
- Der Aufwand bleibt im Rahmen des Projekts realistisch.
- **Nachteile gegenüber einer vollständigen Eigenmodellierung:**
 - Die Simulation stimmt geometrisch nicht vollständig mit dem realen Roboter überein.
 - Gelenkpositionen, Abstände oder Bauteilformen können von der Hardware abweichen.
 - Kollisionen und Reichweiten können in der Simulation anders wirken als in der Realität.
 - Manuelle Korrekturen in Unity können zusätzlichen Aufwand verursachen.
- **Konsequenz:** Die Simulation ist ausreichend für Visualisierung, Lernen und grundlegende Tests, aber nicht für hochpräzise geometrische oder kollisionskritische Analysen geeignet. Die Abweichungen der CAD-Modelle müssen als bekannte Einschränkung dokumentiert werden.

10.8. ADR 8: Keine Simulation der Roboterkamera

- **Kontext:** Der reale MyPalletizer kann mit einer Kamera verwendet werden. Diese Kamera ist jedoch stark von realen Umgebungsbedingungen abhängig, zum Beispiel Beleuchtung, Perspektive, Bildrauschen, Fokus, Kalibrierung und realen Objekteigenschaften.
- **Entscheidung:** Die Kamera des realen Roboters wird in der Unity-Simulation nicht vollständig nachgebildet. Die Simulation konzentriert sich auf Roboterbewegung, Werkzeuge, LED-Steuerung und Weltobjekte.
- **Vorteile gegenüber einer simulierten Kamera:**
 - Der Entwicklungsaufwand bleibt deutlich geringer.
 - Die Simulation bleibt fokussiert auf die Kernfunktionen des Projekts.
 - Es müssen keine komplexen Kameramodelle, Bildverarbeitung oder Kalibrierungsdaten nachgebildet werden.
 - Fehlerquellen durch unrealistische Kamerabilder werden vermieden.
- **Nachteile gegenüber einer simulierten Kamera:**
 - Kamerabasierte Anwendungen können nicht vollständig in der Simulation getestet werden.
 - Bildverarbeitung, Objekterkennung oder visuelle Kalibrierung müssen weiterhin mit realer Hardware geprüft werden.
 - Die Simulation bildet nicht alle möglichen Einsatzszenarien des Roboters ab.
 - Der Digital Twin ist in Bezug auf sensorisches Feedback eingeschränkt.
- **Konsequenz:** Die Simulation eignet sich für Bewegungsabläufe, Werkzeugverhalten und Szenarien mit bekannten Objekten. Kamerabasierte Funktionen werden jedoch nicht abgedeckt und müssen separat mit der realen Hardware getestet werden.

11. Quality Requirements

Die Qualitätsanforderungen machen die wichtigsten Ziele des Systems anhand konkreter Szenarien messbar. Sie beschreiben, welche Eigenschaften das System im Betrieb erfüllen soll und welche Architekturentscheidungen diese Ziele unterstützen.

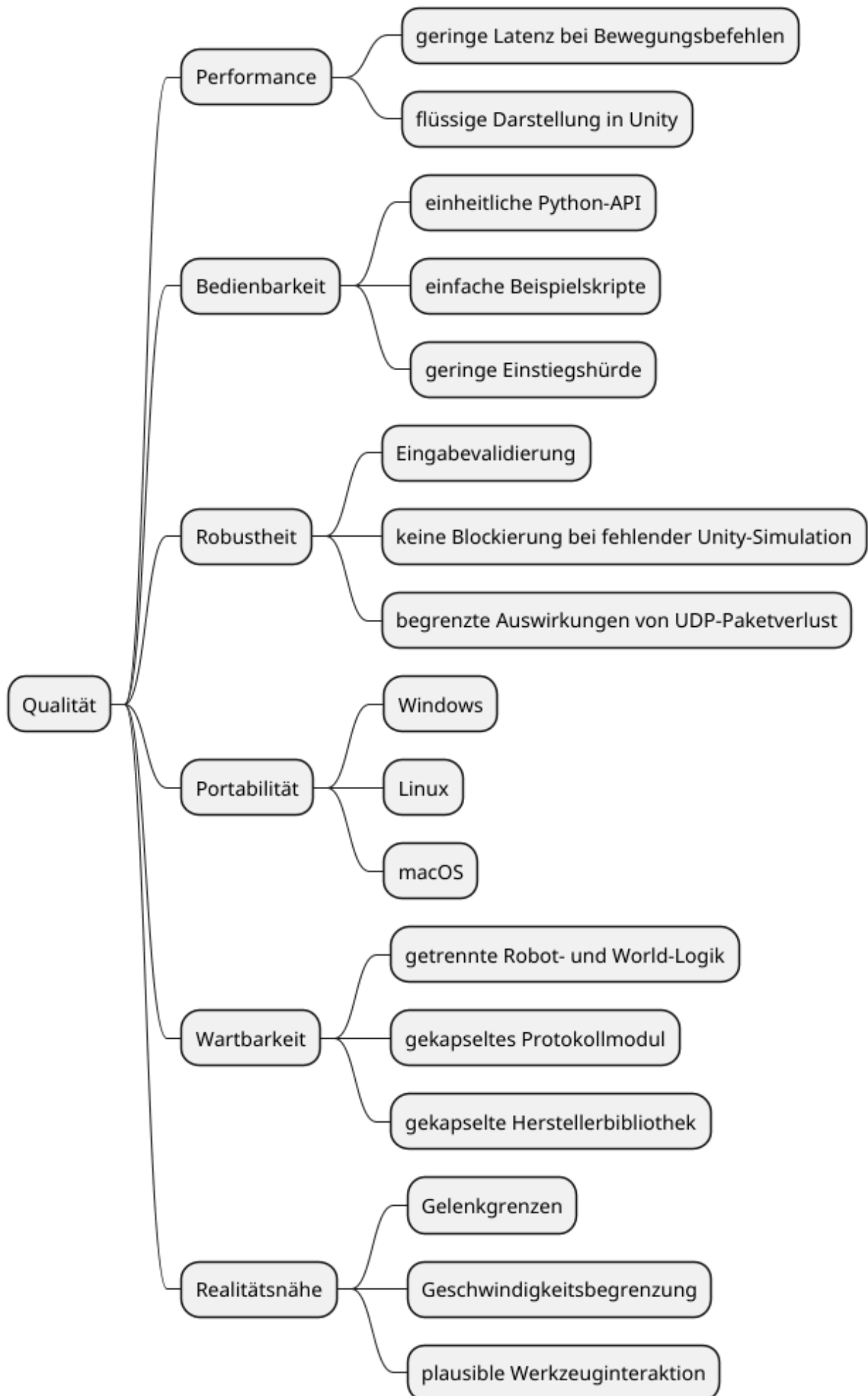
11.1. Quality Requirements Overview

Für dieses System sind vor allem eine schnelle Reaktion, eine einfache Nutzung sowie ein robuster Betrieb wichtig. Da das System sowohl in der Lehre als auch für Tests verwendet wird, spielen zudem Portabilität, Wartbarkeit und Nachvollziehbarkeit eine wichtige Rolle.

Qualitätsmerkmal	Priorität	Begründung
Performance	Sehr hoch	Bewegungsbefehle sollen in der Simulation schnell sichtbar werden, damit sich die Steuerung direkt und natürlich anfühlt.
Bedienbarkeit	Hoch	Die Zielgruppe besteht vor allem aus Studierenden und Robotik-Interessierten. Die API muss daher einfach verständlich und schnell nutzbar sein.
Portabilität	Hoch	Das System soll auf typischen Rechnern mit Windows, Linux oder macOS lauffähig sein.
Robustheit	Hoch	Fehler in der Simulation oder Kommunikation sollen das Python-Skript möglichst nicht unkontrolliert abbrechen lassen.
Realitätsnähe	Hoch	Die Simulation soll das Verhalten des realen Roboters plausibel abbilden, auch wenn sie keine vollständig physikalisch exakte Nachbildung ist.
Wartbarkeit	Mittel	Neue Befehle, Presets oder Werkzeuge sollen ergänzt werden können, ohne die Grundarchitektur umzubauen.
Erweiterbarkeit	Mittel	Das System soll später um weitere Weltobjekte, Werkzeuge oder Kommunikationsfunktionen erweitert werden können.
Nachvollziehbarkeit	Mittel	Nachrichten, Zustände und Fehler sollen für Entwicklung und Debugging verständlich bleiben.

11.2. Quality Tree

Die Qualitätsziele lassen sich wie folgt strukturieren:



11.3. Quality Scenarios

ID	Qualitätsziel	Szenario	Akzeptanzkriterium
QS-1	Performance	Ein Benutzer ruft <code>send_angles(...)</code> im Modus <code>virtual</code> auf. Die Simulation soll innerhalb kurzer Zeit sichtbar auf den Befehl reagieren.	Die Bewegung in Unity beginnt im Normalfall innerhalb von spätestens 50 ms , sofern Unity bereits läuft und die UDP-Kommunikation lokal erfolgt.
QS-2	Portabilität	Ein Benutzer wechselt von einem Windows-System auf macOS oder Linux. Dasselbe Python-Skript soll weiterverwendet werden können.	Das Skript benötigt keine Codeänderungen, sofern Python, die Projektabhängigkeiten und bei Bedarf die passende Unity-Version installiert sind.
QS-3	Robustheit	Die Unity-Simulation wird während der Ausführung eines Python-Skripts beendet.	Das Python-Skript darf durch das Senden von UDP-Nachrichten nicht dauerhaft blockieren. Es soll weiterlaufen oder einen nachvollziehbaren Fehler liefern.
QS-4	Eingabevalidierung	Ein Benutzer sendet einen Gelenkwinkel ausserhalb des erlaubten Bereichs.	Der Wert wird vor der Weitergabe an Simulation oder Hardware auf den zulässigen Bereich begrenzt.
QS-5	Bedienbarkeit	Ein neuer Benutzer installiert die Software und führt ein Beispielskript aus.	Innerhalb von etwa 15 Minuten soll eine erste Bewegung in der Simulation erfolgreich gestartet werden können.
QS-6	Wartbarkeit	Eine neue Nachricht für die Simulation soll ergänzt werden, zum Beispiel ein neuer Weltbefehl.	Die Erweiterung kann im <code>protocol</code> -Modul und in der zuständigen Unity-Komponente ergänzt werden, ohne bestehende Roboter- und Weltbefehle grundlegend umzubauen.

ID	Qualitätsziel	Szenario	Akzeptanzkriterium
QS-7	Realitätsnähe	Ein Benutzer führt dieselbe Gelenkbewegung in der Simulation und auf dem realen Roboter aus.	Die Bewegungsrichtung, Gelenkreihenfolge und grobe Endposition sollen plausibel übereinstimmen. Kleine Abweichungen durch CAD-Modell, Physik und Laufzeiten sind zulässig.
QS-8	Fehlertoleranz	Ein einzelnes UDP-Paket geht während einer Bewegung verloren.	Das System soll nicht abstürzen. Bei zustandsbasierten Bewegungsbefehlen kann ein späteres Paket den Zielzustand wieder korrigieren.
QS-9	Sicherheit	Ein Benutzer gibt eine zu hohe Geschwindigkeit oder ungültige RGB-Werte ein.	Die Werte werden vor der Ausführung auf erlaubte Bereiche begrenzt, damit keine offensichtlich ungültigen Befehle an Simulation oder Hardware gesendet werden.
QS-10	Nachvollziehbarkeit	Während der Entwicklung tritt ein Fehler in einer JSON-Nachricht oder in einem Weltbefehl auf.	Die Nachrichtenstruktur ist durch v und type eindeutig genug, um den Fehler im Protokoll oder in der zuständigen Unity-Komponente gezielt einzugrenzen.
QS-11	Erweiterbarkeit	Es soll ein neues Simulationsobjekt ergänzt werden, zum Beispiel ein weiterer geometrischer Körper.	Das neue Objekt kann über die Objektstruktur und den WorldSpawnManager ergänzt werden, ohne die Robotersteuerung zu verändern.
QS-12	Reproduzierbarkeit	Ein Dozent oder Benutzer möchte ein Testszenario erneut ausführen.	Durch Presets kann dieselbe Ausgangssituation mehrfach geladen werden, ohne alle Objekte manuell neu zu erstellen.

11.4. Hinweise zu Zielkonflikten

Einige Qualitätsziele stehen teilweise im Konflikt zueinander:

- **Performance vs. garantierte Zustellung** UDP wurde gewählt, um niedrige Latenz zu erreichen. Dafür wird keine garantierte Zustellung einzelner Pakete geboten.
- **Realitätsnähe vs. Entwicklungsaufwand** Eine vollständig physikalisch exakte Simulation wäre deutlich aufwendiger. Das Projekt priorisiert daher eine plausible und lernorientierte Simulation.

12. Risks and Technical Debts

Dieses Kapitel beschreibt mögliche Risiken sowie technische Kompromisse, die im aktuellen System bestehen. Die Risiken betreffen sowohl Architekturentscheidungen als auch konkrete Implementierungsdetails.

12.1. Architektur-Risiken

Risiko	Erläuterung und Minderung
UDP ohne garantierte Zustellung	Da UDP keine garantierte Zustellung bietet, können einzelne Pakete verloren gehen. Minderung: Das Protokoll überträgt bei Bewegungsbefehlen jeweils den vollständigen Soll-Zustand. Dadurch kann ein späteres Paket einen verlorenen Zustand wieder korrigieren.
Unterschiedliches Verhalten von Simulation und Hardware	Im Modus both kann es vorkommen, dass Simulation und realer Roboter nicht exakt gleich schnell reagieren oder sich leicht unterschiedlich bewegen. Minderung: Beide Systeme verwenden dieselben Zielwerte und denselben speed -Parameter. Für eine genauere Synchronisation wäre in Zukunft ein Rückkanal mit Ist-Werten sinnvoll.
Port-Konflikte	Die verwendeten UDP-Ports 5005 und 5006 könnten auf einem System bereits belegt sein. Minderung: Die Ports sind konfigurierbar und können bei Bedarf angepasst werden.
Open-Loop-Architektur	Die Python-Seite sendet Befehle, ohne aktiv zu prüfen, ob diese in der Simulation oder auf der Hardware exakt wie erwartet ausgeführt wurden. Minderung: Die Komplexität bleibt dadurch geringer. Für zukünftige Versionen wäre ein Feedback-Mechanismus mit Status- und Fehlermeldungen sinnvoll.
Einseitige Kommunikation zur Simulation	Die Kommunikation zur Simulation erfolgt aktuell hauptsächlich von Python nach Unity. Dadurch erhält die Python-Seite keine detaillierten Laufzeitinformationen aus Unity zurück. Minderung: Für zukünftige Erweiterungen könnte ein Rückkanal ergänzt werden, zum Beispiel für aktuelle Gelenkwinkel, Objektzustände oder Fehlerfälle.
Abhängigkeit von Unity	Die Simulation ist stark an Unity und C# gebunden. Ein Wechsel auf eine andere Simulationsumgebung wäre mit erheblichem Aufwand verbunden. Minderung: Die Kopplung wird reduziert, indem Python über JSON/UDP kommuniziert und nicht direkt von Unity-internen Klassen abhängig ist.

12.2. Implementierungs-Risiken und technische Schulden

- **Begrenzte Modellgenauigkeit** Die Simulation bildet den realen Roboter nur näherungsweise ab. Werte wie Bewegung, Reibung, Greifverhalten oder Interaktionen mit Objekten sind ausreichend für Tests, Demonstrationen und Lernzwecke, aber nicht für hochpräzise physikalische Analysen.
- **Ungenaue oder abweichende CAD-Modelle** Die vom Hersteller bereitgestellten CAD-Modelle stimmen nicht vollständig mit dem realen Roboter überein. Dadurch kann es zu geometrischen Abweichungen zwischen Simulation und Hardware kommen, zum Beispiel bei Abmessungen, Gelenkpositionen oder Bauteilformen. Dies beeinflusst die visuelle Genauigkeit der Simulation und kann dazu führen, dass Bewegungen oder Kollisionen in Unity nicht exakt dem Verhalten des realen Roboters entsprechen. Eine mögliche Minderung besteht darin, die CAD-Modelle manuell zu überprüfen, anzupassen oder bei Bedarf eigene vereinfachte Modelle zu erstellen, die näher an der realen Geometrie liegen.
- **Nicht simulierte Roboterkamera** Die am realen Roboter vorhandene Kamera wird in der aktuellen Simulation nicht nachgebildet. Dadurch können kamerabasierte Funktionen, Bildverarbeitung oder visuelle Rückmeldungen nicht vollständig in Unity getestet werden. Tests, die von Kamerabildern oder realen visuellen Informationen abhängen, müssen deshalb weiterhin mit der echten Hardware durchgeführt werden. Eine mögliche spätere Verbesserung wäre die Integration einer virtuellen Kamera in Unity, die ähnliche Perspektiven und Bilddaten wie die reale Roboterkamera liefert.
- **Abhängigkeit von `pymycobot`** Der Modus `real` hängt von der externen Herstellerbibliothek `pymycobot` ab. Änderungen, Inkompatibilitäten oder Fehler in dieser Bibliothek können direkte Auswirkungen auf die Hardwareansteuerung haben. Eine mögliche spätere Verbesserung wäre eine stärkere Entkopplung durch eine eigene Hardware-Abstraktionsschicht.
- **Threading in Unity** UDP-Nachrichten werden ausserhalb des normalen Unity-Update-Zyklus empfangen. Wenn Unity-Objekte direkt aus einem Hintergrundthread verändert würden, könnte dies zu instabilem Verhalten führen. Deshalb werden Nachrichten zunächst in eine Queue gelegt und anschliessend im Unity-Hauptthread verarbeitet. Diese Lösung reduziert das Risiko, erfordert aber sorgfältige Pflege der Bridge-Komponenten.
- **Manuelle Systemtests** Ein Teil der Systemtests wird manuell durchgeführt. Dadurch können Testszenarien zwar praxisnah überprüft werden, sie sind aber weniger reproduzierbar als automatisierte Tests. Eine mögliche technische Schuld besteht darin, dass Regressionen bei Änderungen an Python oder Unity nicht automatisch erkannt werden. Langfristig wäre eine stärkere Automatisierung der Systemtests sinnvoll.

13. Glossary

Begriff	Definition
Digital Twin	Ein digitaler Zwilling ist ein virtuelles Abbild eines physischen Objekts. In diesem Projekt bezeichnet der Begriff die Unity-Simulation des MyPalletizer 260 M5, die Bewegungen, Werkzeuge und einfache Szenarien nachbildet.
MyPalletizer 260 M5	Der reale Roboterarm von Elephant Robotics, der in diesem Projekt simuliert und optional über die Python-API angesteuert wird.
Unity	Eine Entwicklungsumgebung und Engine zur Erstellung der 3D-Simulation. In diesem Projekt wird Unity für Visualisierung, Gelenkbewegung, Werkzeugverhalten und Weltverwaltung verwendet.
Articulation Body	Eine Unity-Komponente zur physikalischen Simulation von kinematischen Ketten. Sie wird verwendet, um die Gelenkstruktur des Roboterarms stabiler und nachvollziehbarer abzubilden.
UDP	User Datagram Protocol. Ein verbindungsloses Netzwerkprotokoll, das für geringe Latenz geeignet ist, aber keine garantierte Zustellung einzelner Pakete bietet.
JSON	JavaScript Object Notation. Ein textbasiertes Datenformat zum Austausch strukturierter Nachrichten zwischen Python und Unity.
End-Effektor	Werkzeug am Ende des Roboterarms. In diesem Projekt werden insbesondere Greifer und Vakuumpumpe betrachtet.
Preset	Vordefiniertes Szenario für die Simulationswelt. Presets ermöglichen wiederholbare Ausgangssituationen für Tests, Übungen und Demonstrationen.
CAD-Modell	Dreidimensionales Modell eines Bauteils oder Roboters. Im Projekt dienen CAD-Modelle als Grundlage für die visuelle Darstellung des Roboters in Unity.
Open-Loop-Steuerung	Steuerungsansatz, bei dem Befehle gesendet werden, ohne dass ihre erfolgreiche Ausführung aktiv über einen Rückkanal bestätigt wird.
APPLAI	Applied Artificial Intelligence. Modul an der Hochschule Luzern, in dessen Rahmen das Projekt entwickelt wurde.
AI-ROBOTICS	Modul an der Hochschule Luzern, in dem das Projekt weiterentwickelt wird.
HSLU	Hochschule Luzern.